PPT-LAT-Roadmap

Project: shannon-prime-lattice **Document role:** Operational roadmap. Read by every future session before doing work. **Status:** Living document. Mutable. Papers are scaffolding, not artefacts. **Last rewrite:** 2026-05-21 **Authors:** Knack + Claude + Gemini (Shannon-Prime team)

---

0. Read-this-first

This document supersedes the prior linear v1 roadmap that staged work as KSTE → engine → multi-node. That framing was too narrow. The project now spans four parallel engine backends, four (and growing) model families, inline weight and KV-cache compression as **foundational** (not an opt-in feature), and the Lattice blockchain track running in parallel with the inference work.

Two binding rules govern every session that picks up this roadmap:

1. **Anti-contamination rule.** Do **not** read, copy, vendor, or reach into source files under `D:\F\shannon-prime-repos\shannon-prime\` or `D:\F\shannon-prime-repos\shannon-prime-engine\`. Those repositories exist for archival reference only. The math papers under `D:\F\shannon-prime-repos\papers\PPT-ARM\` are **conceptual reference** — read them for theory, never paste code from them. This project rebuilds everything from scratch. Cross-pollination has already produced regressions in this organization at least ten times in six months. The memory item `feedback_no_cross_contamination` is load-bearing here.
2. **Papers are scaffolding, not artefacts.** PPT-LAT-Theory.md, PPT-LAT-Systems.md, and this file are the lattice from which the system is built, not finished publications. Sessions may amend any of them when reality contradicts the design. The blockchain protocol spec in particular is expected to evolve.

If those two rules feel restrictive, that is the point. The previous sessions that ignored them produced the artefacts collected in the `feedback_no_cross_contamination` and `feedback_dont_frankenpatch` memory items.

---

1. Why the scope expanded

The original v1 roadmap presumed a single hot path: build KSTE, wire it into one engine, demonstrate dedup, layer blockchain on top. Three realities forced the expansion:

- **Backend reality.** Real users sit on RTX 3000/4000, on Apple Silicon via Vulkan-MoltenVK, on Snapdragon phones with Hexagon DSP and HTP V69, and on Linux servers with AVX-512. A single-backend prototype cannot be promoted to a production system. Each backend has its own build environment, kernel idioms, and bring-up bugs. The roadmap treats them as four parallel tracks because trying to serialize them costs months for no architectural gain.
- **Model-family reality.** Llama 3.x, Qwen3.0–3.7, Gemma 2.5/3/4, and DeepSeek V4 do not share enough surface area to let one model carry the lattice features alone. They differ in RoPE shape, attention grouping, FFN topology (dense vs MoE), normalisation, and tokenisation. The roadmap therefore plans for an explicit model-family × backend matrix (a 7-row × 4-column grid in Phase 3), and assumes that filling this matrix is the project's centre of mass.

- **Compression-foundation reality.** Inline Q8 weight storage with Frobenius scale, Q4 mixed-precision under calibration, and VHT2 + Möbius + Spinor KV-cache compression are not "features that may be enabled later." They are the **load-bearing memory layout** that makes long-context, multi-model, multi-backend inference fit at all. The roadmap treats compression as bedrock, not topping.

The Lattice features themselves — KSTE dedup, ARM gossip aggregation, dominance verification, two-token economy, blockchain protocol — layer on top of the compressed-inference stack and are off-by-default behind environment-variable gates until they are individually proven.

---

2. Phase summary table

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
0	Bootstrap	Repos, papers, env scripts, workspace	DONE	DONE
1	Math core foundations	O_K, CRT-NTT, poly-ring, VHT2, Frobenius, KSTE built and tested in isolation	All T_* unit tests green on Win+Linux	3–4 weeks
2-CPU	Engine, CPU backend	Reference forward pass + compressed weights + NTT attention on x86	E_CPU_1..E_CPU_6 green	**CLOSED 2026-05-22**
2-CU	Engine, CUDA backend	Same scope as 2-CPU on NVIDIA GPU	E_CU_1..E_CU_6 green	**CLOSED 2026-05-22**
2-VK	Engine, Vulkan backend	Same scope as 2-CPU on cross-platform GPU	E_VK_1..E_VK_6 green	**CLOSED 2026-05-23**
2-HX	Engine, Hexagon backend	Same scope as 2-CPU on Snapdragon HTP V69	E_HX_1..E_HX_6 green	**ESSENTIALLY CLOSED 2026-05-23 (formal tag pending E_HX_5/E_HX_6)**
2-FMT	Engine, .sp-model on-disk format	Loader + transcoder + round-trip gate	E_FMT_1..E_FMT_4 green	**CLOSED 2026-05-23**

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
2-L1	L1 ABI implementation in mathcore	RELOCATE → VALIDATE → HANDLE → SESSION	Each sub-phase has its own gate; umbrella `lat-phase-2-l1-closed`	RELOCATE done; VALIDATE next
3	Model-family expansion	All four backends host all seven model families	M_×*B*_ matrix green	8–12 weeks
4	Inline cache compression validated	PPL drift and memory savings measured per backend × model	Drift ≤ 1% on calibrated families	4 weeks
5	Lattice features (sieve, ARM, dominance)	Off-by-default ENV-gated overlays	Regression suite green when gates off	6 weeks
6	Two-node CRT-sharded inference demo	Dual-prime forward pass across two boxes	End-to-end demo run	3 weeks
7	DHT crawler skeleton	Kademlia-like routing, single-node first	Two-node lookup works	3 weeks
8	Position-as-Arithmetic crawl assignment + **Fibonacci-Prime DHT**	2-axis prime-lattice (semantic) × φ-hashed (load) key space	Deterministic re-assignment + uniform load under skewed inputs	2 weeks + 1 day (Fibonacci hashing)

Phase	Title	Goal in one line	Gate (must pass before advancing)	Wall-clock estimate
9	ARM gossip aggregation + **Golden-Ratio key init**	Capacity-bounded HRR merge across peers; φ-spaced phases replace random projection	Capacity curve extends past prior K=64 ceiling	3 weeks + 2 days (key init)
10	Verification layer	Spot-check via ≤_d, slashing simulator	Slashing simulator stable	4 weeks
11	Two-token economy simulator	Discovery vs work-token dynamics	Equilibrium observed in simulator	3 weeks
12	Blockchain protocol design	Protocol scaffolding; simulator-only first	Simulator passes 1k-block run	6 weeks
13	End-to-end pilot	3 nodes, full stack, real metrics	Pilot report delivered	4 weeks

Wall-clock estimates assume one solo developer plus AI agents working a standard week. They are **planning numbers**, not commitments. The phase gates are non-negotiable. The estimates are.

2.1 How to read the phase table

A few notes on interpreting the table above for sessions picking up the project cold:

- The phases are not strictly serial. Phase 2's four backends are four parallel tracks. Phase 3's matrix is up to 28 parallel cells. Sections 3 and 20 below detail which work can overlap.
- A **gate** is a binary go/no-go check. If a gate fails, the phase is not closed and no later phase that lists it as a dependency may start. The estimates are not gates and may slip without changing the go/no-go status.
- The phase numbering survives reorganisation. If a future session decides Phase 8 should split into 8a and 8b, the table grows vertically; existing phase numbers do not shift. This is what makes the `lat-phase-<n>-closed` tags durable.

2.2 The big picture in one paragraph

The project rebuilds the Shannon-Prime mathematical machinery from scratch (Phase 1), wires it into four engine backends (Phase 2), spreads it across the in-scope model families (Phase 3), validates inline compression end-to-end (Phase 4), layers the Lattice-specific content-dedup features on top behind environment gates (Phase 5), demonstrates two-machine sharded inference (Phase 6), grows the distributed-system layer (Phases 7–9), adds verification and economy simulation (Phases 10–11), specifies the protocol (Phase 12), and runs a three-node pilot

(Phase 13). Every phase has a contract; sessions that close a phase write an offload note and tag the repository.

---

3. Anti-contamination, contracts, and offload pattern

3.1 Anti-contamination

Every Phase 1 file is created from a blank buffer. The math papers under `papers/PPT-ARM/` may be consulted for theorem statements; their implementations may not be copied. Sessions that violate this rule have historically produced (a) silent format drift between the new repo and the legacy one, (b) duplicate names that confuse later searches, and (c) regressions when a "shared" file in the legacy repo is updated and the new repo silently inherits a behaviour change.

If a session feels the urge to look at the legacy code "just for reference," the correct move is to read the theorem statement in the paper and re-derive the implementation from scratch. The derivations are short. The bugs that come from cross-contamination are long.

3.2 Contract system

Every phase below lists:

- **Goal** — one paragraph.
- **Deliverables (file list)** — every file the phase must produce.
- **Tests (named, gated)** — every test the phase must pass, named so that future sessions can search for the exact identifier.
- **Entry conditions** — what must be done before starting.
- **Exit conditions** — what must be true to close the phase.
- **Dependencies** — which other phases gate this one.
- **Notes for the picking-up session** — what to read first, what gotchas past sessions have hit.

A phase is closed only when every deliverable exists, every test passes on both Windows and Linux (where applicable), and an offload note has been written.

3.3 Offload pattern

Each working session ends by writing `papers/SESSION-STATE-lat-<phase>.md` with: current phase, files touched, tests passing, tests failing, next concrete action, any open questions. Sessions that pick up the project read this file first, then this roadmap.

When a phase closes, the corresponding session-state file is renamed to `SESSION-CLOSED-lat-<phase>.md` and a one-paragraph summary is added to this roadmap's "Phase log" appendix.

3.4 VSCode workspace + build environments

The workspace at `D:\F\shannon-prime-repos\shannon-prime-lattice.code-workspace` opens the four repositories the project actually touches:

- `shannon-prime-system` — math primitives.
- `shannon-prime-system-engine` — backend implementations.
- `shannon-prime-lattice` — Lattice features + this roadmap.
- `shannon-prime-lattice-node` — DHT / blockchain node code (later phases).

Build environment scripts live under `shannon-prime-system-engine/scripts/env/`:

- `env-cpu-msvc.bat` — Windows MSVC 19.x, AVX2/AVX512 flags pinned.
- `env-cpu-gcc.sh` — Linux gcc 12+, -march=native baseline.
- `env-cuda.bat` — VS2019 Build Tools + CUDA 12.4, `--use-local-env` flag, target SM 75/86/89.
- `env-vulkan.bat` — Vulkan SDK 1.3.x + glslc.
- `env-hexagon.bat` — Hexagon SDK 5.x on Windows host, Git sh.exe prepended to PATH per the `reference_hexagon_build_recipe` memory.

Versions are **pinned** in each script. A session that needs to bump a toolchain bumps the script in a dedicated commit and notes the change in the offload file.

3.5 GitHub workflow

All four repositories are private. Workflow is per-phase push, no pull requests required (solo developer plus agents). Tags are cut at each phase close, named `lat-phase-<n>-closed`. Commits inside a phase use the prefix `[lat-<phase>]` so the log is grep-able.

3.6 Testing discipline

Every phase keeps the **full regression suite green**, not just its own tests. This is the rule that prevents Phase 5+ Lattice features from silently breaking Phase 2 backends. Sessions land work behind a flag if they cannot make the regressions hold; turning the flag on is a separate commit that runs the suite again.

3.7 Platform-gate policy (amended 2026-05-21)

The original Phase 1 exit conditions read "all tests pass on Windows MSVC and Linux gcc." That conflates three distinct platforms with three distinct costs, and as written it could not be satisfied in a single session on the current dev host (Windows, no local Linux, MSVC needs a separate toolchain bring-up). This amendment splits the platform gate explicitly. It does **not** weaken any test — every T_* still has to pass everywhere eventually; it only stages *which platform closes when*, and names the follow-up wave so it cannot be silently dropped.

The dev host has MinGW **gcc 15.2** and **MSVC v142 (VS2019 BT)** available; **no local Linux**.

Three-tier close, per Phase 1 subphase:

- **Tier 1 — Windows MinGW-gcc (closes in-session).** Every T_* test passes under MinGW gcc on Windows. This is the primary gate a Phase 1 session closes. Repo tag suffix: `lat-phase-1<X>-gcc-closed`.
- **Tier 2 — Linux gcc (CI).** Verified by the GitHub Actions workflow `.github/workflows/ci.yml` added in the Phase-1 scaffold pass (ubuntu-latest, gcc, `cmake -G Ninja` + `ctest`). Linux is not run locally; the CI run on push is the Linux gate. A subphase is not fully closed until its CI job is green on `origin/main`.
- **Tier 3 — Windows MSVC (follow-up wave).** A dedicated later session configures each module in a second build dir via `vcvarsall x64` and runs `ctest` under MSVC. Repo tag (full close): `lat-phase-1<X>-closed`.

`__int128` and the cross-platform identity tests. The CRT-NTT parity oracle `ntt_ref_int128.c` uses `__int128`, which MinGW gcc and Linux gcc support but **MSVC does not**. The production path (`ntt_crt.c`) is already forbidden `__int128` by T_NTT_5, so this only affects the oracle. Strategy: the oracle runs under gcc and dumps its reference vectors to a checked-in binary fixture (`core/ntt_crt/ntt_ref_vectors.bin`); the MSVC build of the test loads that fixture and compares bytes, rather than recompiling the `__int128` oracle. The same checked-in-fixture pattern serves the other cross-platform identity gates (T_NTT_2/3, T_VHT_5, T_KSTE_4):

the gcc build is the byte-source-of-truth, MSVC and Linux CI compare against it. This makes "bit-identical across platforms" a concrete file-diff rather than a re-derivation.

So for a Phase 1 session on this host, "subphase closed" means **Tier 1 green locally + Tier 2 green in CI**, tagged `-gcc-closed`. Tier 3 (MSVC) is tracked as open in the offload note and closed in its own wave.

---

4. The math primitives — what we are recreating

This section names every math primitive the project will rebuild from scratch and what each one is for. Implementations come in Phase 1.

4.1 O_K integer arithmetic over Q(√-163)

Q(√-163) is one of the nine imaginary quadratic fields with class number 1 (a Heegner number). The ring of integers O_K is a unique factorisation domain. We choose Q(√-163) specifically because:

- UFD means every nonzero non-unit factors uniquely (up to units and order) into irreducibles. That property anchors Möbius reconstruction and lets us treat token indices as products of irreducibles without ambiguity.
- The class group is trivial. No nontrivial ideal class equivalences haunt the storage layout.
- The discriminant -163 fits comfortably in 64-bit arithmetic for the norms we care about.

All exact arithmetic — KSTE node values, Möbius reconstructions, ARM binding accumulators — happens in O_K. The implementation must support add, mul, conjugate, norm, division-with-remainder when the divisor's norm permits, and irreducible factorisation up to a configurable norm bound.

4.2 Polynomial-ring attention over R_q = Z_q[x]/(x^N+1)

R_q is the negacyclic polynomial ring used by CKKS and most modern lattice cryptosystems. We use it as a **replacement for the softmax attention kernel**: queries, keys, and values are encoded as polynomials in R_q; the inner-product step becomes a polynomial multiplication modulo (x^N + 1); the softmax becomes a p-adic exponential on the coefficients followed by a normalisation pass.

Phase 1 targets N = 256 (matching the per-head dimension of the current model targets) and parameterises N over {128, 256, 512} so that all three sizes share a single code path. (N = 1024 was in the original draft but is **not supported** on the frozen CRT primes — see §4.3; the primes admit a primitive 2N-th root only up to N = 512. Amended 2026-05-21.)

4.3 CRT-NTT dual-prime kernel with no __int128

Multiplying in R_q efficiently requires the Number Theoretic Transform. We use a dual-prime CRT decomposition so the kernel never needs 128-bit arithmetic and ports to any 64-bit ALU including Hexagon and Vulkan compute shaders.

- q1 = 1073738753, q2 = 1073732609 (two 30-bit Proth primes). These are **frozen** — the dominance-commitment verification and the DHT key topology (§4.4) derive from these exact prime residues, so they do not change.
- Each prime admits a primitive 2N-th root of unity for **N up to 512**, not 1024. Both have $q-1 = 2^{10} \cdot (\text{odd})$, so the 2-adic valuation is 10: $2N \mid (q-1)$ holds for $2N \le 1024$, i.e. $N \le 512$. A negacyclic NTT at N = 1024 would need $2^{11} \mid (q-1)$, which neither prime satisfies.

The supported ring degrees are therefore {128, 256, 512}. (Amended 2026-05-21 after the original draft over-claimed N = 1024.)
- The combined modulus M = q1·q2 ≈ 2^60 carries the full result without overflow.
- CRT recombination uses Barrett reduction; no 128-bit divide.

Independent NTT branches run per prime. Bit-exactness is checked against a reference 60-bit __int128 implementation that exists **only as a parity oracle** (compiled out of production builds).

4.4 Frobenius lifting for Q8 / Q4 weight storage

Frobenius lifting takes a quantised weight stored as a small integer plus a per-row shift and decompresses it inline at matmul time. The per-row Frobenius scale is the key design choice: a single per-tensor shift collapses to noise on Q4 (the cautionary tale from prior validation), while per-row shifts hold accuracy.

- Q8: 1 byte per coefficient + per-row scale. 8× memory compression of the unquantised arena. Production target.
- Q4: 4 bits per coefficient + per-row scale + calibration table. Mixed-precision path; gated on a per-tensor calibration that may promote outlier rows to Q8.

Decompression is fused into the matmul kernels, not done as a separate pass. There is no decoded scratch arena in production builds.

4.5 VHT2 + Möbius reorder + 63-byte Spinor block

The KV cache is stored as a sequence of 63-byte Spinor blocks. Each block carries:

- A VHT2 (Vilenkin Hierarchical Transform) header that describes which basis vectors the block compresses.
- A Möbius-reordered coefficient body.
- A trailing checksum byte.

The byte layout is **frozen** the moment Phase 1D ships. Future sessions do not adjust it without a compatibility migration plan. The cache file format and the on-wire DHT format both reference this layout, so a silent change breaks every node on the network simultaneously.

4.6 KSTE encoder + Tier-0/Tier-1 dominance

The Knight-Spinor Tree Encoder maps a K-vector into a 64-byte packed tree in T_{60,3}. The encoder is deterministic, lossless under the dominance partial order, and produces a compact signature suitable for hashing.

Tier-0 dominance compares roots. Tier-1 dominance compares the first level of children. The Lattice dedup mechanism uses Tier-0 to bucket content into rough equivalence classes and Tier-1 to confirm a match before counting it as a duplicate. The encoder must produce identical output on every backend so that two nodes can compare signatures without a tie-break protocol.

4.7 ARM — Algebraic Resonance Memory

ARM stores HRR-style key-value bindings in the CRT cyclotomic ring. Binding is a polynomial multiplication; recall is an inverse-binding multiplication followed by a similarity search. The choice of the CRT cyclotomic ring is what lets ARM share its kernel inventory with the poly-ring attention layer: a single NTT-based pointwise multiply serves both. Storing ARM bindings as 63-byte Spinor blocks would in principle work too; in practice we store the polynomial coefficients directly because ARM banks are written-once-read-many and benefit from a flat layout.

Phase 1 ships ARM as a single-node primitive. Phase 9 layers gossip on top so peers can merge their ARM banks with bounded capacity loss. The capacity bound is the usual HRR sqrt-K bound: with K bindings packed into a single ring element of dimension N, recall accuracy degrades as roughly sqrt(K/N). For N = 256 this means a slab holds about 16 high-confidence bindings before recall fidelity drops below 0.9 cosine similarity, which is the threshold the Phase 9 capacity-curve experiment will report against.

4.8 Inline weight compression as a foundation, not a feature

The single most consequential design choice in this roadmap is that inline Q8 (and eventually Q4) weight storage is not an optional optimisation but the **default memory layout**. Every backend's matmul kernel reads packed bytes and applies the per-row Frobenius scale inline; there is no decoded fp32 arena that holds the same weights in expanded form. The reason this matters is twofold:

- Memory pressure. An unquantised arena on Gemma3-1B already costs 10.4 GB; the Q8 packed form is 1.3 GB. The unquantised arena does not fit on the phone path at all, and it costs 8× the network transfer on the multi-node CRT-shard path.
- Bandwidth determinism. Every backend uses the same packed byte layout, so cross-backend verification compares bytes for bytes rather than fp32 approximations. This is what makes the cross-backend determinism test (OQ-8) tractable at all.

The cost of treating compression as foundational is that every backend must implement the packed-byte matmul before any larger model can be brought up. Sessions that attempt to "do correctness first, compression later" hit a wall when the unquantised path runs out of memory on the test rigs and need to backtrack to compression-first anyway.

4.9 Inline KV-cache compression as a foundation

The same logic applies to the KV cache. The Spinor block layout specified in §4.5 is the only KV layout in the production paths. There is no "uncompressed KV cache" mode in production builds; the reference fp32 cache exists for the parity tests only, compiled out of release builds. The reason is identical to the weight-compression case: long-context inference does not fit on any of the target backends without KV compression, and the CRT-shard and DHT paths both consume the on-disk and on-wire byte layout directly, so the production format has to match from day one.

---

5. The 13-step Prime Power Transformer (canonical table)

The replacements below are the canonical algebraic interpretation of every step in a transformer forward pass under the PPT framework. They are reproduced verbatim from Paper I §10 so future sessions can grep for the exact wording.

Step	Operation	Algebraic replacement
1	Embedding lookup	Möbius reconstruction over squarefree token indices; CRT vocabulary sharding
2	RMSNorm (pre-attn)	Mersenne-prime scaling; Poncelet closure $d^2=R^2-2Rr$
3	Q/K/V projections	Twin-prime head pairing; sexy-prime 6:1 GQA grouping
4	SP Write (KV → archive)	Poncelet closure as eviction trigger; CRT-sharded KV

Step	Operation	Algebraic replacement
5	FUSED_KQ	UFD-exact decompression; Heegner endomorphism
6	Softmax	p-adic exponential on integers; circulant attention on closed orbits
7	Fused V weighted sum	Spinor reconstruction across twin-paired heads
8	Attention output projection	CRT decomposition of W_O into independent sub-matrices
9	FFN (skeleton + residual)	Mersenne-dimensional skeletons; n^2+n+41 cold-start
10	Activation oracle update	Cramér prime-gap prefetch; Poncelet early exit
11	Residual add + norm	Group-law residual on E(K)
12	Per-layer loop	$n\delta\equiv 0$ adaptive depth; caustic projection
13	LM head	CRT pruning of vocabulary logits; Mersenne-prime sampling

This is the operational mapping from "what the model does" to "what we build." Every Phase 2 backend ultimately wires these thirteen steps into its forward pass. Every Phase 3 model family threads the same table through its specific topology.

---

6. Theorems and extensions on the anchor

Treat the following as the "already proven" set. Sessions do not need to re-prove them; they need to call out when an implementation contradicts them.

- **T1 — Endomorphism realization.** A hidden-state trajectory through L transformer layers embeds in the L-fold product of an elliptic curve E over O_K. Layer composition is endomorphism composition.
- **T2 — Möbius UFD compression.** Exact reconstruction over O_K is possible at the squarefree basis. This is the theoretical anchor for the squarefree token index in step 1.
- **T3 — Hasse–Weil = Shannon limit.** The Hasse–Weil bound $|E_p(F_p) - (p+1)| \leq 2\sqrt{p}$ coincides with the Shannon-channel capacity bound applied to the CM curve. The two limits are the same constant to within a unit.
- **T4 — Frobenius cancellation.** On Gemma3-1B, the Frobenius lift is bit-identical to six significant figures. The original validation produced PPL 13.11 against a baseline of 13.12 (delta 0.08%). That single-number figure was the original acceptance criterion; under the production **per-row** Frobenius arena (E_CPU_9, ~1% lossy by design) it is **superseded** by the split T_FRO_4 gate of §8.2 — forward correctness (engine-f32 vs the f16 oracle $\leq 0.05\%$) separated from per-row Q8 quality (drift vs engine-f32 $\leq 2\%$). See the §8.2 T_FRO_4 closure clause.
- **T5 — Deuring / CM Sato-Tate.** The Frobenius trace a_p is asymmetrically distributed between primes that split and primes that remain inert in O_K. This asymmetry is what makes class-number-1 fields work for compression — the inert primes carry more bits per symbol.
- **T6 — CRT exact sharding.** The dual-prime kernel is bit-identical to the 60-bit reference on Linux gcc and Windows MSVC. Portability across any 64-bit ALU is implied.
- **E9.1 — Stern-Brocot RoPE.** Discrepancy $\varphi = 0.00134$ against 0.05576 for standard RoPE. The Stern-Brocot positional encoding is a strict improvement on quasirandomness grounds.

- **E9.2 — Weil pairing on E[n].** Miller's algorithm validated; bilinearity confirmed empirically. The pairing math is solid; the open research question is the embedding from hidden states into E[n].
- **E9.3 — Hecke multiplicativity.** 20 of 20 random trials confirm the Hecke operator multiplicativity at composite levels.
- **E9.5 — LLL reduction.** 20 of 20 trials confirm LLL-reduced bases produce smaller KV-write footprints than the naive basis.
- **E9.6 — BSD analytic rank.** Toy curves verified via Sage. Not load-bearing for the production system; included for completeness.
- **E10 — Iwasawa μ=0.** Residual-stream depth stability follows from the μ=0 conjecture in the relevant Z_p-extension. The empirical evidence is consistent with μ=0 on all curves tested.

When a Phase 2 backend produces a result inconsistent with one of these theorems, the theorem is correct and the implementation is wrong. That is the working assumption until proven otherwise.

The reason this rule reads so strongly is operational: every time a prior session has assumed the math was off, the math has turned out to be right and the bug has been in the storage layout, the kernel ordering, or a sign convention. The theorems above have been validated to six significant figures on real hardware running real models; the chance that a fresh backend implementation is the first place those theorems fail is vanishingly small compared to the chance that the implementation has a transposed index somewhere. The corollary is that when a backend disagrees with another, the older backend is the ground truth and the newer one is the suspect.

A second corollary: do not bend the math to make the implementation easier. There is a recurring temptation, especially on the Phase 2-HX track, to drop the dual-prime CRT in favour of a single 30-bit prime because "the test corpus fits anyway." Resisting that temptation is the whole reason Phase 1B's T_NTT_5 grep gate exists. The dual-prime kernel is not an optimisation. It is the substrate Phase 6's two-node demo runs on, the substrate Phase 8's position-as-arithmetic key derivation reads from, and the substrate the verification layer audits. Collapsing it to single-prime to save bring-up time costs multiple later phases.

---

7. Phase 1 — Math core foundations

Goal. Stand up every math primitive in section 4 as a standalone library inside `shannon-prime-system`, with deterministic unit tests that run on Windows MSVC and Linux gcc.

Dependencies. Phase 0 (DONE).

Subphases run in parallel. Phase 1A through 1F are independent modules. A session can pick any one without blocking the others. The recommended priority order — for a serial reader — is 1A → 1B → 1D → 1C → 1E → 1F, because the polynomial-ring and Frobenius layers benefit from the CRT-NTT kernel being already in place, and KSTE wants the VHT2 block format frozen first.

Why these particular primitives. The Phase 1 list is not a buffet of mathematical curiosities. Each primitive answers a specific question about how the production system stores and moves bits:

- O_K answers "how do we factor token indices uniquely so Möbius reconstruction works without ambiguity?" The class-number-1 property is the load-bearing piece.
- The dual-prime CRT-NTT answers "how do we multiply in the cyclotomic ring on every backend without needing 128-bit arithmetic?"

- Polynomial-ring attention answers “how do we replace softmax with something that lives in the same algebraic universe as the cache representation, so the kernel inventory shrinks?”
- The Spinor block answers “what is the on-disk and on-wire byte layout of a single KV record?” The 63-byte frozen layout is the contract.
- Frobenius lifting answers “how do we store weights at 1 byte per coefficient and still get bit-identical PPL?”
- KSTE answers “how do we produce a content signature that two nodes can compare bit-for-bit without a tie-break protocol?”

Implementing all six in parallel is what makes Phase 1 fit in a few weeks rather than a few months. Sessions that try to serialise will discover that the bottleneck is rarely arithmetic — it is build environments, harness wiring, and offload notes. Spreading the implementation work across parallel subphases keeps any single bottleneck from gating the whole phase.

7.1 Phase 1A — O_K arithmetic over Q(√-163)

- **Deliverables.** `system/ok/ok_int.h`, `ok_int.c`, `ok_int_test.c`.
- **Tests.**
  - T_OK_1 — UFD verification on 256 random norms ≤ 2^20.
  - T_OK_2 — Norm and conjugate identities (N(αβ) = N(α)N(β), conj(conj(α)) = α).
  - T_OK_3 — Multiplication closure (every product lands back in O_K with correct integer coordinates).
  - T_OK_4 — Ideal class triviality: every fractional ideal of norm ≤ 2^16 is principal.
  - T_OK_5 — Round-trip from rational integer to O_K and back.
  - T_OK_6 — Irreducible factorisation for 1024 random norms ≤ 2^14.
- **Entry conditions.** None.
- **Exit conditions.** All six tests pass on Win+Linux. No undefined behaviour under UBSan.
- **Notes.** The Heegner number 163 is non-negotiable. Other Heegner numbers exist (1, 2, 3, 7, 11, 19, 43, 67, 163) but only 163 has the discriminant we want.

7.2 Phase 1B — CRT-NTT dual-prime kernel

- **Deliverables.** `system/ntt/ntt_crt.h`, `ntt_crt.c`, `ntt_crt_test.c`, `ntt_ref_int128.c` (parity oracle, compiled out of production).
- **Tests.**
  - T_NTT_1 — Forward/inverse round-trip on 4096 random polynomials at each N ∈ {128, 256, 512}.
  - T_NTT_2 — Bit-exactness vs `ntt_ref_int128` on Linux gcc.
  - T_NTT_3 — Bit-exactness vs `ntt_ref_int128` on Windows MSVC.
  - T_NTT_4 — Pointwise multiply followed by inverse equals coefficient-wise modular multiplication of the original polynomials (negacyclic convolution).
  - T_NTT_5 — No 128-bit type used in the production path (static assert in header, plus a grep gate in CI).
- **Entry conditions.** None.
- **Exit conditions.** All five tests pass.
- **Notes.** Barrett reduction for the CRT step. Twiddle factors precomputed and cached per N. The parity oracle is a regression anchor for the lifetime of the project — keep it building even when it is not in the production path.

7.3 Phase 1C — Polynomial-ring attention R_q

- **Deliverables.** `system/poly/poly_ring.h`, `poly_ring.c`, `poly_ring_test.c`.
- **Tests.**

  - T_PR_1 — Polynomial multiply matches schoolbook reference at N ∈ {128, 256, 512} for 1024 random polynomial pairs each.
  - T_PR_2 — Inner-product attention KL ≤ 1e-7 versus softmax baseline at d = 256 (matches the prior Gemma3 head-size result on the legacy repo).
  - T_PR_3 — Negacyclic property: x · p(x) wraps as expected (the last coefficient becomes the negated leading coefficient).
  - T_PR_4 — Cross-N stability: the same input expressed at N=256 and N=512 (zero-padded) produces the same result up to the shared coefficient prefix.
- **Entry conditions.** Phase 1B closed.
- **Exit conditions.** All four tests pass.
- **Notes.** The KL bound T_PR_2 is the closest thing the project has to a soft acceptance criterion for "the math is right." Take it seriously.

7.4 Phase 1D — VHT2 + Möbius reorder + 63-byte Spinor block

- **Deliverables.** `system/spinor/spinor_block.h`, `spinor_block.c`, `vht2.c`, `mobius_reorder.c`, `spinor_test.c`. Frozen byte layout specified in `papers/PPT-LAT-Systems.md` §3.
- **Tests.**
  - T_VHT_1 — Block round-trip: encode then decode identical bytes for 65,536 random vectors.
  - T_VHT_2 — Möbius reorder bijection: every permutation is a true permutation (no element collisions).
  - T_VHT_3 — 63-byte total size verified by `sizeof(spinor_block_t)`.
  - T_VHT_4 — Checksum byte detects any 1-bit corruption in the body.
  - T_VHT_5 — Endianness: identical bytes produced on little-endian Linux and Windows hosts.
  - T_VHT_6 — Layout frozen flag: a `LAYOUT_VERSION` constant in the header must be incremented by hand if any field moves, and CI refuses a layout change without a migration plan file.
- **Entry conditions.** None.
- **Exit conditions.** All six tests pass; layout reviewed and signed off in the offload note.
- **Notes.** The 63-byte total is deliberate — 64 minus the checksum, packed tight, no padding. Future sessions: do not add a "convenient" reserved byte.

7.5 Phase 1E — Frobenius lift for Q8 weight storage

- **Deliverables.** `system/frobenius/frobenius_lift.h`, `frobenius_lift.c`, `frobenius_test.c`.
- **Tests.**
  - T_FRO_1 — Per-row scale picker correctness against the ceiling-shift reference (which exists from prior validation; reimplemented from scratch under the anti-contamination rule).
  - T_FRO_2 — Encode-then-decode round-trip max error ≤ 1 ULP for a fixed test matrix.
  - T_FRO_3 — 8× memory reduction confirmed on a 4096×4096 weight matrix.
  - T_FRO_4 — On Gemma3-1B, Frobenius-lifted weights produce PPL matching the f16 oracle (the T4 acceptance check repeated under new code). **DEFERRED to Phase 2** (amended 2026-05-21): T_FRO_4 requires a working forward pass + a loaded Gemma3-1B, neither of which exists in Phase 1 (the engine lands in Phase 2). **CLOSED in Phase 2-CPU 2026-05-22** via the split gate (forward-correctness ≤ 0.05% vs oracle; per-row Q8 drift ≤ 2%) — see the §8.2 closure clause; the original single "within 0.1%" target was superseded because the production arena is per-row (~1% lossy). Phase 1E closes on the pure-math tests T_FRO_1..3.
- **Entry conditions.** Phase 1A closed.

- **Exit conditions.** T_FRO_1..3 pass under UBSan; T_FRO_4 deferred to Phase 2 (now CLOSED there 2026-05-22 via the split gate — see the §8.2 closure clause).
- **Notes.** Q4 is **not** in Phase 1E. Q4 lives in Phase 4 because it needs calibration tables that depend on a working forward pass.

7.6 Phase 1F — KSTE encoder + Tier-0/Tier-1 dominance

- **Deliverables.** `system/kste/kste_encode.h`, `kste_encode.c`, `kste_dominance.c`, `kste_test.c`.
- **Tests.**
  - T_KSTE_1 — Encoder determinism: identical K-vector produces identical bytes 1,000,000 trials.
  - T_KSTE_2 — Tier-0 dominance partial-order axioms: reflexivity, antisymmetry up to equivalence, transitivity.
  - T_KSTE_3 — Tier-1 confirmation rule: if two trees pass Tier-0 and Tier-1, they share the same first-level child set.
  - T_KSTE_4 — Cross-platform identity: encoder produces identical bytes on Windows MSVC and Linux gcc.
  - T_KSTE_5 — 64-byte packed-tree size enforced.
- **Entry conditions.** None.
- **Exit conditions.** All five tests pass.
- **Notes.** Tier-0/Tier-1 nomenclature comes from the original KSTE paper. Future sessions sometimes try to "simplify" to a single tier for ergonomic reasons; the two-tier check is what makes the Lattice dedup adversarially robust. Do not collapse it.

7.7 Phase 1 exit

Phase 1 closes when all six subphases close, the regression runner prints six green sections, and `SESSION-CLOSED-lat-1.md` is written. The repository tag is `lat-phase-1-closed`.

7.8 Pseudocode sketch — Phase 1B kernel skeleton

The following pseudocode captures the shape of the dual-prime CRT-NTT kernel without descending into a real implementation. It exists here to anchor the API a Phase 1B session is expected to land.

```
struct ntt_ctx {
    uint32_t N;             // ring degree, in {128, 256, 512}
    uint32_t q1, q2;        // two 30-bit Proth primes
    uint32_t* psi1; uint32_t* psi2;   // 2N-th roots of unity per prime
    uint32_t* inv_psi1; uint32_t* inv_psi2;
    // Barrett mu values for each prime
};

ntt_ctx* ntt_init(uint32_t N);
void   ntt_forward (const ntt_ctx*, const int32_t* in, uint32_t* out1, uint32_t* out2);
void   ntt_inverse (const ntt_ctx*, const uint32_t* in1, const uint32_t* in2, int64_t* out);
void   ntt_pointwise_mul(const ntt_ctx*, const uint32_t* a1, const uint32_t* a2,
                                         const uint32_t* b1, const uint32_t* b2,
                                           uint32_t* out1, uint32_t* out2);
void   ntt_crt_recombine (const ntt_ctx*, const uint32_t* x1, const uint32_t* x2, int64_t* ou
```

The contract is: `ntt_forward` produces two residue vectors; pointwise multiply happens per residue (no cross-prime arithmetic); `ntt_inverse` applies the inverse transform per residue and `ntt_crt_recombine` reassembles a signed 60-bit result via Barrett reduction. No 128-bit

type appears anywhere. The same kernel is reused by 1C (poly-ring attention), 4.7 (ARM), and 1E (Frobenius matmul) without modification.

7.9 Pseudocode sketch — Phase 1D Spinor block

```
#define LAYOUT_VERSION 1u    // bumping requires migration plan
struct spinor_block_t {       // 63 bytes total, packed
    uint8_t  vht2_header[7];
    uint8_t  mobius_body[55];
    uint8_t  checksum;        // CRC-8 of header || body
};
_Static_assert(sizeof(spinor_block_t) == 63, "frozen layout");
```

Encoders and decoders must round-trip on every backend; the cross-platform identity test T_VHT_5 specifically compares bytes byte for byte between Linux gcc and Windows MSVC builds.

---

8. Phase 2 — Engine backend tracks (CPU, CUDA, Vulkan, Hexagon)

Goal. Each backend supports a single reference model end-to-end: GGUF load → forward pass → token-level output. Frobenius-lifted weights are inline-decompressed. NTT-attention is wired but the **sieve is OFF**. Inline KV-cache compression is wired but the **Lattice features are OFF**. The reference model is Qwen3-0.6B Q8.

Dependencies. Phase 1 closed.

Parallelism. The four backends are independent. Sessions can run 2-CPU and 2-CU concurrently with no shared state. 2-VK and 2-HX can start the moment their build environments are sorted, regardless of 2-CPU and 2-CU status. The matrix below shows the per-backend subphase plan — same letter-coded structure across all four.

8.1 Per-backend subphase template

For each backend B ∈ {CPU, CU, VK, HX}:

- **2-B.A — GGUF loader.** Parse the GGUF header, materialise tensors into backend-native storage. No SP transforms applied yet.
- **2-B.B — Forward pass on Qwen3-0.6B Q8.** Reference correctness vs llama.cpp logits on the same prompt. Acceptance: max-token logit difference ≤ 1e-4 absolute or ≤ 0.1% relative.
- **2-B.C — Frobenius-quantised matmul with inline decompression.** Production memory layout. 8× compression confirmed end-to-end.
- **2-B.D — Backend-specific vectorisation.** AVX2 for CPU, AVX512 optional second pass, CUDA tensor cores where applicable, Vulkan subgroup ops, HVX vectorisation for Hexagon.
- **2-B.E — NTT-attention path wired in.** Sieve OFF. PPL must remain within Phase 1C's T_PR_2 tolerance.
- **2-B.E.1 — Lossless Polynomial-Shift RoPE Cache (the production spec).** Precondition: SP_ENGINE_NTT_ATTN=1 (poly-ring attention active). Mechanism: inside the negacyclic cyclotomic ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$, a RoPE rotation by angle $\Delta \cdot \theta_d$ becomes a

discrete polynomial index shift modulo N — no continuous trigonometry, no cos / sin table. The relative-attention rotation cache collapses from $O(\text{ctx} \cdot D)$ fp32 values to $O(\text{ctx})$ integer shift offsets. For $D = 128$, ctx $= 4096$: from $4096 \times 128 = 524{,}288$ floats (2 MB) down to 4096 int32 offsets (16 KB). **~128× memory reduction, mathematically lossless on stock geometric-RoPE models.** Applies to Gemma3, Qwen3, Llama 3.x and every other model the engine targets — no φ-RoPE precondition, no frequency-schedule swap, no PPL-drift trade.

Acceptance: T_PR_2 / E_CPU_5 already measures the gate. The NTT polynomial-shift representation is exact inside the ring up to int32 quantisation; E_CPU_5 on Qwen3-0.6B has it at KL $\approx 2.7 \times 10^{-10}$ mean against the fp32 reference. Activating the shift-cache representation does not introduce any additional error beyond what E_CPU_5 already gates. Per-backend gates inherit the same property: when the backend's NTT-attention path passes E_B_5 / T_PR_2, the polynomial-shift cache is in-spec by construction.

Why this works on stock RoPE. The Three-Gap Theorem (T7) is about *linear* irrational sequences $\{k\alpha\}$; geometric RoPE $\theta_d = \text{base}^{-2d/D}$ does not satisfy that precondition, so the original "≤3 distinct adjacent gaps across dimensions" framing did not apply. But the cyclotomic-ring representation bypasses the frequency-axis sort entirely — inside R_q, every rotation is already a discrete index permutation regardless of the underlying θ_d distribution. The win is structural to the ring, not to the frequency schedule.

Gate: built into `SP_ENGINE_NTT_ATTN=1`. When NTT-attention is active, the polynomial-shift cache is the production representation; when NTT-attention is off, the engine falls back to the fp32 rotation table. The previous `SP_ROPE_THREE_STEP=1` / `SP_ROPE_3STEP_CACHE=1` / `SP_ROPE_PHI=1` gate proposals are retired in favour of NTT-coupling.

The φ-RoPE schedule swap and the Three-Gap-derived frequency-sort cache restructuring are moved to **§20 Research Track** at the bottom of this document. They remain interesting for a future φ-RoPE-trained or fine-tuned model but are not on the Phase 2 critical path.

- **2-B.F — KSTE-encoded KV cache.** Gated by SP_KSTE_KV=1. Off by default. Phase 2 only requires that the encode path produces identical signatures across backends and that decode reproduces the same KV.

Tests E_B_1..E_B_6 mirror the six subphases. The numbering follows the subphase letter: E_CPU_1 covers 2-CPU.A, E_CPU_2 covers 2-CPU.B, and so on.

Amended 2026-05-22: the per-backend template is extended with the foundational-compression items first specified for CPU in §8.2.1 — **G: Q4 inline weight compression** (E_B_7), **H: inline VHT2+Spinor KV codec** (E_B_8), and the **persistent-KV O(n) decode loop** (GEN_B). Each later backend (2-CU/VK/HX) mirrors E_B_7/E_B_8/GEN_B alongside E_B_1..E_B_6, with the same gates (`SP_ENGINE_FROB=3`, `SP_KV_SPINOR=1`, `qwen3_generate_kv`) defaulting OFF and the cross-backend tolerance (output vs CPU within 1e-3). Do not silently drop them. See §8.2.1 for the CPU definitions and gates.

Three-Gap deliverables across phases (added on the T7 anchor in PPT-LAT-Theory): the Three-Gap Theorem (T7) unifies four optimisations that land in different phases of the roadmap but share one substrate (golden-ratio rotation in phase space). Cross-reference summary so a session picking up any one of these knows where the others live:

- **2-B.E.1 — Lossless Polynomial-Shift RoPE Cache** (production spec, all backends). RoPE rotation in the cyclotomic ring R_q is a discrete polynomial index shift mod N. Rel-attn cache collapses from $O(\text{ctx} \cdot D)$ fp32 values to $O(\text{ctx})$ int32 offsets — ~128× memory

reduction. Gated by SP_ENGINE_NTT_ATTN=1. Lossless on stock RoPE models; acceptance is the same E_B_5 / T_PR_2 gate that already closes the NTT-attention path (no separate gate needed).

(φ-RoPE frequency swap and the original Three-Gap frequency-sort cache restructuring are demoted to §20 Research Track.)

- **Phase 4 amendment — Fibonacci KV sub-sampling.** When KV cache exceeds memory bounds, retain tokens at positions $\lfloor k\varphi \cdot N \rfloor \mod N$ instead of FIFO/LRU. Bounded discrepancy on the temporal axis. Gated by SP_KV_FIB=1. Validates against the PPL-drift sweep already run for VHT2+Spinor.
- **Phase 8 amendment — Fibonacci-Prime DHT** (1-day deliverable under Phase 8). 2-axis address space: prime-factored lattice for semantic adjacency (axis 1) × Fibonacci hashing for load balance (axis 2). Solves Kademlia clustering natively without cryptographic hashing. See PPT-LAT-Systems §4.4.
- **Phase 9 amendment — ARM Golden-Ratio key init** (2-day deliverable under Phase 9). Replace random projection + Gram-Schmidt with deterministic φ-spaced phases in R_q. Capacity curve expected to extend past the prior K=64 ceiling at 0.15 cosine recall. See PPT-LAT-Systems §4.2.
- **§6.x amendment — Validator selection by Golden-Ratio rotation** (rolls into Phase 11/12 — Two-token economy and Blockchain). Replace VRF/randao beacon with deterministic $\{b\varphi\}$ partitioning of the stake-weighted validator set. Provably fair, no consensus rounds needed. See PPT-LAT-Systems §6.x.

All five items default OFF / additive. None are blocking dependencies for the foundational compression and model-family work — they are optimisations that layer on once the underlying compression and DHT infrastructure is green.

8.2 Phase 2-CPU (the canonical track)

STATUS: CLOSED 2026-05-22. All six E_CPU gates + T_FRO_4 split-gate green. Engine commits through `3431cf8` (SP2 `41c5e19` SentencePiece encode/decode, SP3 `3431cf8` `sp_perplexity` + `test_ppl`). T_FRO_4: engine-f32 vs f16 oracle `−0.0146%` (gate ≤ 0.05%); per-row Q8 arena drift `−0.74%` (gate ≤ 2%). Full CPU regression 20/20 green incl. T_FRO_4. Tag `lat-phase-2-cpu-fro4-closed`, `lat-phase-2-closed`. Offload: `SESSION-CLOSED-lat-2-CPU.md`.

Original spec, retained for historical context:

- **Build env.** `scripts/env/env-cpu-msvc.bat` (Windows) and `scripts/env/env-cpu-gcc.sh` (Linux).
- **Reference model.** Qwen3-0.6B Q8.
- **Tests.**
  - E_CPU_1 — GGUF loader round-trips header bytes.
  - E_CPU_2 — Forward pass reproduces the stock-llama.cpp **distribution** on identical token IDs (see §8.6.1 for why a per-logit tolerance is the wrong gate). Three properties, default pure-f32 path:
    (a) top-1 argmax agrees at every position; (b) ggml's top-1 lies inside the engine's top-5 and vice versa at every position; (c) mean KL(`ggml || engine`) over positions < 1e-5 nats (measured ~2.3e-6 on Tier-1; override via SP_KL_MAX).
  - E_CPU_3 — Frobenius/Q8 weight path. (a) **Lift faithfulness** (the "identical logits to a reference fp32 matmul"): the inline-lift matmul (SP_ENGINE_FROB=1, accumulate `q·x` then scale once) and the dequant-then-f32-dot of the *same* Q8 weights (SP_ENGINE_FROB=2) agree to float-associativity (max |Δlogit| < 1e-2; measured ~1e-4). (b) **Q8 quality** vs the engine's own pure-f32 path (NOT ggml, §8.6.1): mean

KL(f32||q8) below a regression gate (default 5e-2; measured ~2e-2). Per-row Frobenius Q8 is lossy by design (one scale per wide row, vs ggml's per-32-block Q8_0), so it legitimately flips a few low-margin argmaxes — argmax is reported, not gated. Real PPL quality is T_FRO_4.
  - E_CPU_4 — AVX2 matmul (8-wide FMA, dot_f32) vs the scalar reference (SP_CPU_SCALAR=1): argmax agreement at every position + max |Δlogit| below the float-reassociation floor (default 1e-3; measured ~1.7e-4). The original "1e-6 elementwise" is unachievable on *final* logits — FMA and 8-wide accumulation reassociate the dot, and QK-RMSNorm amplifies that over 28 layers exactly as in §8.6.1 (1e-6 would only hold per single matmul output). AVX512 shares the gate when built.
  - E_CPU_5 — NTT-attention (SP_ENGINE_NTT_ATTN=1, sieve OFF): each attention score <q,k> is recovered EXACTLY as coefficient 0 of the negacyclic poly-ring product (sp_pr_inner, head_dim=128=ring N) after int32 quantization (scale 2^16) of the post-norm/post-RoPE head vectors; softmax + V-sum stay f32. Gate: argmax agreement + mean end-to-end KL(softmax-baseline||ntt) ≤ 1e-7 (the literal T_PR_2). Measured ~2.7e-10 mean — because the inner product is *exact* (only int32 quant deviates), this meets the tight T_PR_2 bound end-to-end, unlike the F16/FMA gaps of E_CPU_2/E_CPU_4.
  - E_CPU_6 — KSTE KV-cache overlay (qwen3_forward_ex with a kv_trees sink; production gate SP_KSTE_KV=1), sieve OFF, encoder only. KSTE is a one-way deterministic encoder (no decode), so the "round-trip identical bytes" is byte-identical determinism + store/load: each post-norm/post-RoPE K head-vector (int32-quantized) is encoded to its 64-byte signature, and the test asserts (1) two independent prefills produce byte-identical signatures, (2) every signature survives a store/load unchanged and carries the frozen wire form (version/branching/depth/reserved + k=head_dim), (3) distinct K vectors give distinct signatures. Measured on Qwen3-0.6B: 6944 signatures (28×31×8), all deterministic + wire-valid.
- **Notes for picking up.** CPU is the canonical track because it has the fewest build dependencies. Bring up CPU first if any other backend is blocked; using CPU outputs as the ground truth for the other backends is built into the test harness.

T_FRO_4 — Gemma3-1B PPL gate (CLOSED 2026-05-22, SP3 commit 3431cf8). The §6 / T4 acceptance check, recomputed under the production code. The original single "PPL within 0.1% of baseline" target conflated two independent things and is **superseded** by a split gate — per-row Frobenius Q8 is ~1% lossy *by design* (E_CPU_3, one scale per wide row vs ggml's per-32-block Q8_0), so 0.1% was never the right bound for the quantised path:

- **(a) forward correctness** — engine pure-f32 PPL vs the stock-llama.cpp f16 oracle on the same corpus + n_ctx, gated at the §8.6.1 precision floor (SP_PPL_GATE_F32, ≤ 0.05% rel). This is the real "Gemma3-1B forward correct" check. Measured **−0.0146%** (engine 32.865 vs oracle 32.869).
- **(b) per-row Q8 arena quality** — Q8 PPL drift vs the engine's own f32 PPL (SP_PPL_GATE_Q8, ≤ 2% rel; judged against the engine's f32, never ggml, per §8.6.1). Measured **−0.74%**.

The Q8 pass reloads via the packed arena (SP_ARENA=q8, byte-identical to SP_ENGINE_FROB=1 per E_CPU_9 but quantised once at load, so the gate runs in minutes); the f32 pass clears all matmul knobs first so a contaminated shell cannot silently turn it into a q8-vs-q8 false PASS. sp_perplexity follows perplexity.cpp: non-overlapping n_ctx chunks, per-chunk BOS re-anchor, score [n_ctx/2, n_ctx-1), PPL = exp(mean NLL); the single-window fixture (n_ctx=168 == token count) matches the dump_logits oracle exactly. SLOW-labelled (its own ctest run, ~158 s); the 176 MB regenerable oracle .ref.bin is gitignored. Full regression **20/20 green incl. T_FRO_4**. This clause is the authoritative T_FRO_4 gate; it supersedes the original single-number "within 0.1%" target (the §6 T4 / §7.5 figure).

8.2.1 Foundational-compression extension (amended 2026-05-22) The original six E_CPU items closed 2026-05-22 (`lat-phase-2-closed`). The inline weight + KV compression that §1/§4.8/§4.9 call **foundational** is only partly covered by them: E_CPU_3 is Q8 weights, E_CPU_6 is the *KSTE* KV overlay (a Lattice-sieve signature encoder, one-way) — neither is **Q4 weight storage** nor the **VHT2+Spinor KV codec** that §4.5/§4.9 freeze as the production KV layout. This amendment adds the missing foundational items as explicit Phase-2-CPU contract gates, plus the persistent-KV decode loop. All are gated OFF by default (regression invariant: gates off ⇒ bit-identical to the E_CPU_2 f32 path). Per §8.6.1 every quality gate is measured against the engine's own pure-f32 logits, never ggml and never the F16-act path.

- **E_CPU_7 — Q4 inline weight compression.** Frobenius Q4: per-row scale, symmetric 4-bit codes in `[-7,+7]` (the Q8 `[-127,127]` rule at 4 bits; two codes per byte), dequant `w_hat = q·s/7`. **Mixed-precision / calibration:** a per-tensor pass promotes high-error ("outlier") rows to Q8. v1 calibration is **weight-only per-row sensitivity** (promote rows whose Q4 round-trip error / row-L2 exceeds a threshold) — the **activation-based** calibration of §4.4 stays a **Phase-4** refinement (§7.5), with the promotion-mask hook left in place. Gate `SP_ENGINE_FROB=3`. Validated on Qwen3-0.6B: lift faithfulness (inline-Q4 matmul == dequant-then-f32-dot of the same Q4 weights, float-assoc) + Q4 quality vs pure-f32 (mean KL under a regression gate, looser than Q8 since Q4 is lossier by design; argmax reported, not gated).
- **E_CPU_8 — Inline VHT2+Spinor KV-cache compression.** The §4.5 frozen 63-byte Spinor block carries 55 int8 anchors; a Qwen3 head vector is head_dim=128, so each post-norm/post-RoPE K and V head vector is stored as ⌈128/55⌉ = **3 Spinor blocks** (balanced 43/43/42 split; the frozen layout is NOT modified). Gate `SP_KV_SPINOR=1`; attention reads the decoded (lossy) K'/V'. Gate OFF ⇒ bit-identical to E_CPU_2. ON: mean KL(f32-KV ‖ spinor-KV) under a regression gate, argmax reported. (Distinct from E_CPU_6's KSTE overlay — this is the foundational lossy KV codec, KSTE is the sieve signature.)
- **GEN_KV — persistent-KV O(n) decode loop.** A KV cache stores position-finalized (post-RoPE) K/V so decode steps append one token without re-rotating. Gate: **argmax-identity** with the O(n²) `qwen3_generate` (greedy sequence equals greedy sequence) under `SP_CPU_SCALAR=1` — NOT bit-equal logits, because the incremental and re-prefill paths reduce different-length softmax sums and so legitimately differ by the float-reassociation floor (§8.6.1). Composes with `SP_KV_SPINOR` (the cache may hold Spinor blocks). Token count `SP_GEN_KV_N` (default 8).

8.2.2 Load-bearing layout: packed-weight arena + persistent KV (amended 2026-05-22) E_CPU_3/7/8 prove the codecs but ran them as a per-matmul *demonstration* (re-quantize on the fly; KV stored f32 with a lossy round-trip). §4.8/§4.9 require the compression to be the **actual memory layout** — and the GPU backends must read the *same packed bytes* (§4.8 "compare bytes for bytes"), so the packed formats are built and frozen on the canonical CPU track **before** 2-CU. The Q4 quant/pack primitive was first lifted into `core/frobenius` (shared by all backends; math-core `T_FRO_Q4`). Three pieces, dependency-ordered:

- **E_CPU_9 — packed-weight arena (Piece 1a, DONE).** `SP_ARENA=q8|q4` (default off): at load, quantize the matmul weights once into a per-row Frobenius Q8/Q4 packed arena; the matmul lifts inline from the codes. Versioned in-memory layout (`SP_ARENA_LAYOUT_VERSION=1`) = the byte format the GPU backends read (per-ROW Frobenius, NOT ggml per-32-block Q8_0). Tight gate: `SP_ARENA=q8` forward **byte-identical** to `SP_ENGINE_FROB=1` (and q4 to =3). Measured: Q8 arena 574.5 MB, Q4 arena 300.7 MB (2.85% rows promoted). Scope 1a: matmul weights only; embedding+norms stay f32 from the mapping (still held).
- **E_CPU_10 — release the F16 source (Piece 1b, DONE).** `SP_ARENA_EMBED=1` folds the embedding into the arena; `qwen3_release_source()` copies norms to owned f32 and `gguf_release_data()` unmaps the GGUF data (keeping the parsed tensor/kv structs).

The forward then reads only the arena + owned norms — peak memory drops from the ~1.5 GB F16 mapping to the packed footprint (~574 MB Q8). SP_ARENA_RELEASE=1 does it at load. Tokenizer owning mode (sp_tokenizer_load_ex(g,1)) copies vocab+merges so it survives the unmap. Gate: forward after release **byte-identical** to held (a dangling pointer would diverge/crash) + gguf_tensor_data NULL post-release + owning tokenizer still decodes. **T_FRO_4 is now CLOSED (see the §8.2 closure clause)** — the arena is the production layout it validates (Gemma3-1B PPL via the split gate; the 2nd arch + SentencePiece it needed both landed in SP2/SP3).

- **Piece 2 — persistent Spinor KV cache (DONE).** qwen3_generate_kv with SP_KV_SPINOR=1 now stores the cache as sp_spinor_block_t[] (NBLK = ceil(head_dim/55) blocks/head; 3 for Qwen3) and decodes on read into a per-LAYER f32 scratch — resident KV memory is the packed blocks + one layer of scratch, not the full f32 cache. decode(encode(x)) is arithmetically the same as the in-place round-trip, so the block cache is **sequence-identical** to the f32 round-trip parity reference, exposed as SP_KV_SPINOR_REF=1 (the §4.9 "fp32 cache for parity tests only"). **Gate GEN_KV_SPINOR** (in test_gen_kv) asserts that identity. **Measured drop: 2.71×** (block 2.96 MB vs f32 8.03 MB on Qwen3-0.6B, P=44) — the honest figure for the *frozen* 63-byte / 55-anchor block at head_dim=128 (3 blocks). The earlier "~6×" was aspirational; the frozen layout gives 512 B → 189 B per head = 2.71×. Larger drops would require changing the frozen block (out of scope; would bump SP_SPINOR_LAYOUT_VERSION).
- **Piece 3 — composability + format-lock (DONE). COMPOSE** gate (test_compose): with the arena ACTIVE (SP_ARENA=q8), the Spinor-block KV cache is still sequence-identical to the f32 KV reference — i.e. the weight source (arena Q8) and the KV layout (blocks vs f32) are orthogonal axes that compose without interference (all three gates on: SP_ARENA + SP_KV_SPINOR + qwen3_generate_kv). **Format-lock:** file-scope _Static_asserts freeze the cross-backend byte contract — arena: SP_FROB_ARENA_LAYOUT_VERSION==1, Q8 qmax 127 / Q4 qmax 7 dequant convention, 4-byte f32 row scale; KV: 63-byte block, 55-anchor body, SP_SPINOR_LAYOUT_VERSION==1. A silent layout drift is now a compile error until the version is bumped with a migration.
- **Piece 4 — port the load-bearing primitives into the math core (DONE, 2026-05-22; user direction "more load-bearing in math-core before 2-CU").** The packed-weight arena FORMAT and the multi-block KV head split were initially written in the engine (src/forward/{arena,forward}.c); since they are cross-backend byte contracts (every backend reads the same bytes, §4.8/§4.9), they were ported into shannon-prime-system so CUDA/Vulkan/Hexagon share one implementation rather than re-deriving them. **core/frobenius** (math-core commit 9a4e0ea): sp_frob_packed_tensor (the mixed-precision per-row Q8/Q4 arena layout) + sp_frob_pack_tensor(rows, cols, prec, promote, get_row_cb, ctx, out, *promoted) (callback-based so GGUF reading stays engine-side, no full-tensor f32 spike) + sp_frob_packed_dequant_row + SP_FROB_ARENA_LAYOUT_VERSION (the single source of truth — the engine's duplicate macro was deleted). Gate T_FRO_5. **core/vht2**: sp_spinor_blocks_for / sp_spinor_encode_vec / sp_spinor_decode_vec (the frozen 43/43/42 head split). Gate T_VHT_7 (split is byte-identical to an explicit per-chunk encode/decode). Engine (b00fbf1) bumps the submodule and consumes them; behavior byte-identical (CPU regression 16/16).

The load-bearing layout (arena + persistent Spinor KV + format-lock), and the primitives that realize it, now live in the math core; all four backends will share one implementation. 2-CU may start (user direction 2026-05-22) — its kernels read the SP_FROB_ARENA_LAYOUT_VERSION=1 arena bytes and the SP_SPINOR_LAYOUT_VERSION=1 KV blocks directly. T_FRO_4 (Gemma3-1B PPL) is the production-quality validation of the arena — CLOSED 2026-05-22 (SP3 commit 3431cf8); see the §8.2 closure clause.

8.3 Phase 2-CU (CUDA backend)

STATUS: CLOSED 2026-05-22. Full CUDA gate set GREEN (6/6): CUDA_SMOKE, M_GEMMA3_CUDA (f32+Q8+Q4), M_QWEN3_CUDA (E_CU_1..4), E_CU_5, E_CU_6, T_FRO_4_CU. Engine HEAD `cc1aafd`. E_CU_2 KL 2.33e-6 (== CPU 2.347e-6). E_CU_5 NTT-attention via int64 exact dot (== CPU `sp_pr_inner` on 192/192 random vectors). E_CU_6 KSTE host-encode three-part gate (cross-backend byte-identity structurally unachievable per `reference-kste-cross-backend-gate`). Tag `lat-phase-2-cu-closed`, `lat-phase-2-cu-fro4-closed`. Offload: `SESSION-STATE-lat-2-CU.md`.

Original spec, retained for historical context:

- **Build env.** `scripts/env/env-cuda.bat`. **Pin update (2026-05-22):** the dev host now has **CUDA 13.2** on PATH and the GPU is an **RTX 2060 (sm_75)** — so the 2-CU bring-up targets **CUDA 13.2 + sm_75** (still a §8.3 supported arch), not the old 12.4 pin. `env-cuda.bat` is to be updated to 13.2 (and likely needs the same ASCII/`goto` fix the CPU env scripts got) when 2-CU starts.
- **Reference model.** Same Qwen3-0.6B Q8.
- **Tests E_CU_1..E_CU_6** mirror the CPU set, with the additional per-platform gate that CUDA output must match CPU output within 1e-3 relative on the same prompt. The looser tolerance accounts for expected fused-multiply-add ordering differences.
- **Notes.** Target SM 75/86/89 (RTX 2060/3000/4000 series). Older SMs not supported. Use `--use-local-env` to keep MSVC from injecting unwanted flags.

8.4 Phase 2-VK (Vulkan backend)

STATUS: CLOSED 2026-05-23. T_FRO_4_VK exit gate GREEN. Mirrors closed 2-CU on Vulkan compute via SPIR-V; SPIR-V shaders glslc-compiled at build time, not runtime. Engine HEAD `0c243eca` / merge `99ccb04d` (`lat-2-VK` branch). Gemma3-1B n_ctx=168 via SP_BACKEND=vulkan: (a) vulkan-f32 PPL **32.86458** vs oracle f16 PPL 32.86939 → `−0.0146%` (gate 0.05%) PASS; (b) per-row Q8 arena PPL 32.62294, drift `−0.7353%` (gate 2%) PASS. Tag `lat-phase-2-vk-closed`, `lat-phase-2-vk-fro4-closed`. Offload: `SESSION-STATE-lat-2-VK.md`.

Original spec, retained for historical context:

- **Build env.** `scripts/env/env-vulkan.bat`. Vulkan SDK 1.3.x. glslc for shader compilation.
- **Reference model.** Same Qwen3-0.6B Q8.
- **Tests E_VK_1..E_VK_6** mirror the CPU set, with output vs CPU matching within 1e-3 relative.
- **Notes.** Lower initial priority than CPU/CUDA. The reason Vulkan is in scope at all is Apple Silicon support via MoltenVK and the option of a cross-platform fallback when CUDA is not available. Subgroup ops are essential — the reduction-heavy parts of NTT-attention rely on them.

8.5 Phase 2-HX (Hexagon backend)

STATUS: ESSENTIALLY CLOSED 2026-05-23. HVX-accelerated Gemma3 forward on V69 cDSP matches CPU Q8 at KL 8.9e-11 (HX.3b green). Engine commits through 654c6a68 (`lat-2-HX` branch): HX.0 env+toolchain hard-gate, HX.1 aarch64-android cross- compile + on-phone CPU PPL `−0.0144%`, HX.2-prep 574 MB rpcmem capacity probe, HX.2 FastRPC IDL round-trip `ping(41)→42` on device, HX.3a per-tensor rpcmem upload byte-exact (CRC match) + cDSP scalar f32 matmul bit-exact + scalar gemma3 forward KL 9.19e-11, HX.3b HVX matmul + HVX gemma3 forward worst_rel 7.9e-5 KL 8.9e-11. T_FRO_4_HX met transitively: on-phone hexagon-Q8 PPL **32.62290** vs f16 oracle 32.86939 → `−0.75%` (gate 2%) PASS. Remaining for

the formal `lat-phase-2-hx-closed` tag: E_HX_5 (host int64-dot identity test) + E_HX_6 (host `sp_kste_encode` 3-part gate). Bounded continuation. Offload: `SESSION-STATE-lat-2-HX.md`.

Original spec, retained for historical context:

- **Build env.** `scripts/env/env-hexagon.bat`. Hexagon SDK 5.x on the Knack Windows host. Git sh.exe prepended to PATH. The `reference_hexagon_build_recipe` memory captures the exact recipe; diverging from it has produced multi-session bring-up delays.
- **Reference model.** Same Qwen3-0.6B Q8, with a phone-suitable context length cap.
- **Tests E_HX_1..E_HX_6** mirror the CPU set, with output vs CPU matching within 1e-3 relative.
- **Notes.** This is the highest build-environment-fragility track in Phase 2. Two prior sessions have hit silent fallback bugs when FastRPC rpcmem registration size mismatched the IDL length parameter (`feedback_fastrpc_exact_alloc`). Do not exceed-allocate. Use the freethedsp shim when SP_FREETHEDSP=1; it is opt-in. QNN HTP V69 is the target accelerator for matmul on this platform.

8.6 Phase 2-FMT — Format implementation (parallel sub-phase)

STATUS: CLOSED 2026-05-23. All four E_FMT gates green. `sp_model_load` (mmap zero-copy) + `sp-transcode` (GGUF → .sp-model) + `.sp-tokenizer` extraction + §12.3 round-trip gate all shipped on the `lat-2-FMT` branch (engine commits `1cfb85a2..3e553fcc`). Gemma3-1B round-trip: `bit_exact=YES, worst_abs=0, L2-drift=0.000000%, argmax 8/8`. Qwen3-0.6B cross-arch: identical numbers. Tag `lat-phase-2-fmt-closed`.

Original spec, retained for historical context:

The `.sp-model` byte layout was frozen as Appendix B of PPT-LAT-Systems at tag `lat-phase2-contract-frozen` (commit `e4f78ae`). The format is specified but **not implemented** in the engine — `sp_model_load`, `sp-transcode`, and the §12.3 round-trip gate are all green-field work.

Phase 2-FMT is the sub-phase that implements them. It is structurally **parallel to** the per-backend tracks (2-CPU, 2-CU, 2-VK, 2-HX) rather than serially blocking them, because:

- `.sp-model` and `sp-transcode` are pure CPU code that runs *before* any backend dispatch. They have no dependency on which math backend(s) are wired up.
- T_FRO_4 in each per-backend track runs against the GGUF + in-RAM Frobenius arena path, not against `.sp-model`. Phase 2-CPU has already closed T_FRO_4 (3431cf8) without `.sp-model` existing.
- The §12.3 round-trip gate (GGUF → `.sp-model` → bit-identical logits) is the format's own validation gate, separate from any backend's T_FRO_4.

Therefore 2-FMT can run concurrently with 2-CU, 2-VK, 2-HX, sequenced only by reviewer bandwidth and not by code dependencies. Recommended landing order: **2-FMT closes before 2-VK and 2-HX start**, so those two have a stable on-disk path to test against. 2-CU is already in flight and need not wait.

10.1 Deliverables

- **E_FMT_1 — `sp_model_load` reference implementation.** Pure mmap + header parse + tensor table pointer setup. Zero allocation proportional to tensor data size. Implements the verification path of Appendix B §3 (magic / version / header CRC-32 / tokenizer_hash → SHA-256 vs paired `.sp-tokenizer`). Lives in `shannon-prime-system-engine/src/io/sp_model_load.c`. Maximum 250 LOC; if it grows past that, something is wrong.

- **E_FMT_2 — `sp-transcode` CLI binary (GGUF → `.sp-model`).** Separate binary at `shannon-prime-system-engine/tools/sp_transcode/`. Reads upstream GGUF, dequantizes any quantized tensors to f32, then re-quantizes into the PPT-native dtype space (`0K_Q8` + sibling `FR0BENIUS_SCALE_FP32`, `0K_Q4`, or `SPIN0R63` per the engine's arch-specific dispatch). Owns the per-arch `sp_arch_info` population (arch_id, RoPE base, GQA groups, SWA window, FFN variant, norm variant, tied_embeddings). Writes the 512-byte header
  - sorted-by-name-hash tensor table + 65536-aligned data region. Enforces the spatial-locality constraint of Appendix B §9 (sibling tensors physically adjacent — parent first, then `.scale`).
- **E_FMT_3 — `.sp-tokenizer` extraction.** Pulls the SentencePiece / BPE blob out of GGUF metadata, writes the 128-byte `.sp-tokenizer` header + blob per Appendix B §7. The 4-byte CRC-32 covers bytes [0, 52). Two-file output: `model.sp-model` + `model.sp-tokenizer`, the latter reusable across fine-tunes of the same base.
- **E_FMT_4 — §12.3 round-trip gate.** The format's own T_FRO_4-class validation: load Gemma3-1B via the GGUF path, then via the `.sp-model` path (post-`sp-transcode`), run `sp_prefill_chunk` on the same corpus, assert bit-identical logits in deterministic mode (or ≤ 0.05% drift in production mode, mirroring T_FRO_4 gate (a)). Lives in `tests/test_sp_model_roundtrip.c`. **This is the closure gate for 2-FMT.**

10.2 Build env

Same `scripts/env/env-cpu-msvc.bat` as Phase 2-CPU — 2-FMT is pure CPU code. No CUDA / Vulkan / Hexagon toolchains required. The transcoder binary builds against the same VS2019 BuildTools + Ninja pin as the engine library.

10.3 Exit

`sp-transcode` produces a valid `.sp-model` + `.sp-tokenizer` pair for each in-scope reference model (Gemma3-1B is the bedrock target; Qwen3-0.6B as cross-arch validation). E_FMT_1..4 are all green. Tag `lat-phase-2-fmt-closed` on `shannon-prime-system-engine` and on `shannon-prime-system` if any math-core changes were required.

After this closes, `.sp-model` is the *recommended* (not *required*) load path for the engine in 2-VK and 2-HX bring-ups. 2-CU may continue on the GGUF path until convenient to switch — switching is not a 2-CU closure dependency.

10.4 Sequencing constraints

- **2-FMT depends on `lat-phase2-contract-frozen` (e4f78ae).** ✓ Met.
- **2-FMT does NOT depend on 2-CU closure.** Can start immediately.
- **2-VK and 2-HX should not start before 2-FMT closes.** Soft recommendation, not a hard block — those agents can pick either the GGUF or `.sp-model` load path. The recommendation exists because a stable on-disk format means the backend agent isn't fighting two moving targets simultaneously.
- **`.sp-model` v1 (any breaking change) requires re-tagging `lat-phase2-contract-frozen` → `lat-phase2-contract-v2` and updating Appendix B.** v0 should not need to evolve during Phase 2; if it does, that's evidence the contract was missing a load-bearing field, and the agent surfacing the gap should call it out explicitly before patching the spec.

10.5 Non-goals for 2-FMT v0

- Multi-file sharding (`.sp-model.NNNN-of-MMMM`). Deferred to v1; see Appendix B §11 open questions.
- GPU-direct storage (NVIDIA GDS) ingestion. Phase-2+ optimization.
- Pre-baked ARM bank seeds in the file. v0 sessions initialize ARM empty at `sp_session_create`.
- Tokenizer-blob compression. v0 ships the SentencePiece / BPE blob uncompressed.

8.7 Phase 2-L1 — L1 ABI implementation (math-core consolidation + session surface)

The frozen L1 ABI (Systems Appendix A, tag `lat-phase2-contract-frozen`) specifies the C/Rust boundary as `sp_model` + `sp_session` opaque handles with `sp_prefill_chunk` / `sp_decode_step` / `sp_session_clone` / `sp_session_rewind` / atomic cancel flag. Making that contract real in code is a four-sub-phase journey — the algorithmic core was relocated into the math core; the session-surface construction comes next.

This sub-phase runs **after** Phase 2-CPU/CU/VK/HX closed because the relocation validates against their existing regression suites; it runs **before** Phase 3 (model-family expansion) because Phase 3 calls `sp_session_*` on a `sp_model *` that must exist.

8.7.1 Phase 2-L1.RELOCATE — relocate reference inference path into math-core (CLOSED) STATUS: CLOSED 2026-05-23. Twelve gated increments on the `lat-l1` branch of `shannon-prime-system` relocated the full reference inference path into the math core: GGUF parser + weight-dtype dequant + .sp-model loader half (`core/io_format`) + .sp-model hash primitives + model- representation header + packed-weight arena (T_ARENA 14/14) + GGUF load/free/release lifecycle (T_MODEL) + model-coupled weight-lift kernels (T_FWD_DISPATCH, bit-exact parity) + Qwen3 forward orchestration (T_FORWARD, end-to-end smoke) + Gemma3 reference forward (T_FORWARD, end-to-end smoke) + Qwen3 KV-decode + greedy generate (T_FORWARD 2/2).

System HEAD 222e252c. Engine still has its (now-redundant) copies of the relocated files; the engine's regression suite hasn't yet been re-run against the math-core sources — that is sub-phase 8.7.2's explicit gate.

8.7.2 Phase 2-L1.VALIDATE — engine integration bump (NEXT) Point each backend's CMake at the relocated math-core library, delete the engine's now-redundant copies of every file the lat-l1 relocation moved, run the existing regression: CPU 20/20 incl T_FR0_4, CU 6/6, VK gates, HX HX.0–HX.3b. Heavy cross-repo work but it is the ONLY validation that proves the twelve relocations are bit-exact against four already-closed backends — and the §8.8.1 distributional-gate discipline applies (argmax + top-5 + KL ≤ existing thresholds; no new per-logit tolerance can be required).

Gate. All previously-green backend gates remain green when the backend consumes the relocated math-core sources rather than the engine-side copies. Bisect any regression to the specific relocation increment that introduced it.

Closure tag. `lat-phase-2-l1-validate-closed` on engine + system. This is the prerequisite for sub-phases 8.7.3 and 8.7.4.

8.7.3 Phase 2-L1.HANDLE — `.sp-model` handle adapter into math-core Build the `.sp-model` handle half inside math-core: migrate the GGUF → `.sp-model` transcoder from engine-side (currently in `shannon-prime-system-engine`, where Phase 2-FMT closed it); build the

sp_model_to_qwen3 and sp_model_to_gemma3 adapters that bridge the loader output to the relocated forward path; commit a small .sp-model fixture to math-core test data (Gemma3-1B Q8 or Qwen3-0.6B Q8 — whichever is smaller after transcode).

Gate. Bit-identical logits via the GGUF-load path vs the .sp-model-load path on the same fixture (Systems Appendix B §12.3, the round-trip gate that's separate from T_FR0_4).

Closure tag. lat-phase-2-l1-handle-closed.

8.7.4 Phase 2-L1.SESSION — sp_session ABI surface Construct the frozen session API in math-core: sp_session_create (on const sp_model *m) + sp_session_destroy + sp_session_arch + sp_prefill_chunk + sp_decode_step + sp_session_clone + sp_session_rewind + the atomic-bool cancel flag wiring per Systems Appendix A §5. The algorithmic core lives inside the relocated qwen3_generate_kv (and the corresponding Gemma3 equivalent); the session shape — persistent handle, chunk/step decomposition, clone/ rewind/cancel semantics — is new construction.

Spec discipline. The session ABI takes const sp_model *m. Do NOT build a temporary veneer wrapping the GGUF-loaded qwen3_model directly; that deviates from the frozen entry point. If sub-phase 8.7.3 isn't closed yet, this sub-phase BLOCKS on it.

Gate (first parity). sp_prefill_chunk returns last-position logits bit-exact to the reference forward's last-position logits in deterministic mode. The reference forward is the relocated gemma3_forward / qwen3_forward from sub-phase 8.7.1.

Gate (full). sp_decode_step advances the session-owned KV by one token, producing logits whose argmax-trajectory over a 100-step decode matches the reference greedy qwen3_generate_kv / gemma3_generate_kv output exactly.

Closure tag. lat-phase-2-l1-session-closed → triggers the umbrella lat-phase-2-l1-closed on engine + system.

8.8 Phase 2 exit

Phase 2 closes when at least one backend (CPU is the minimum) has all six E-tests green. Other backends close independently and are tagged lat-phase-2-<backend>-closed. The CPU backend close also produces lat-phase-2-closed since CPU is the canonical anchor.

8.8.1 Why E_CPU_2 is a distributional gate, not a per-logit tolerance The original gate (forward pass "within tolerance" — read as 1e-4 abs / 0.1% rel per logit) is **unachievable** for a correct scalar f32 forward pass against stock llama.cpp, and the reason is structural, not a bug. Established during the 2-CPU.B bring-up (2026-05-22) by isolation testing against the clean oracle (tools/oracle/{dump_logits,dump_layers,rope_check, attn_check}):

- The matmul projection is F16-exact under matched precision: feeding the engine's F16-activation mode (SP_ENGINE_F16_ACT=1, which rounds each matmul's activations to F16 to mimic ggml's vec_dot_type=F16 src1 downcast) reproduces ggml's Vcur to **1.3e-6**.
- RoPE is bit-faithful (≤2e-6 vs ggml_rope_ext at all positions; it is a linear map, fully characterised by rope_check).
- The attention core (scale / softmax / GQA mapping / causal mask / V-weighted sum) reproduces ggml's kqv to **1e-6** when run on ggml's own Q/K/V (attn_check, all layers).
- **Per-head QK-RMSNorm is a precision amplifier.** It divides each head by its RMS; where that RMS is small it magnifies the ~1e-6 matmul-precision floor by 39× (Q) to 477× (K) — measured at layer 0 with identical input. Compounded over 28 layers at the positions where QK-norm RMS is small, this reaches a ~1–2% worst-case per-logit gap.

So the per-logit gap is real, content/position-correlated, and *inherent to any implementation that does not bit-replicate ggml's SIMD F16 arithmetic* — which a portable scalar reference must not be required to do. The argmax is nonetheless exact and the **distributions are nearly identical**: mean KL(ggml‖engine) ≈ 2.3e-6 nats. KL is therefore the correctness metric (it is also what llama.cpp uses for perplexity-style validation), with top-1/top-5 agreement as the structural guard.

Two gates, two purposes, no contamination:

- **E_CPU_2** validates the engine's pure-f32 path against the *reference model* (ggml): argmax + top-5 + KL. The pure-f32 path is used (not F16-act) because it has *lower* KL to ggml than F16-act does.
- **E_CPU_3+** validate alternate kernels (Frobenius/Q8, AVX2, NTT-attn) against the *engine's own* pure-f32 logits — same arithmetic and accumulation order — so a tight per-kernel tolerance stays meaningful and the ggml precision gap never propagates downstream. Quantize against pure-f32, never against F16-act.

PPL-level corollary — T_FRO_4 gate (a). The same two-gate split scales to the perplexity gate that closes §8.2: engine pure-f32 PPL is compared to the stock-llama.cpp f16 oracle at *this* precision floor (≤ 0.05% rel), while the per-row Q8 arena is judged only against the engine's own f32 PPL (≤ 2%), never against the oracle. See the **§8.2 T_FRO_4 closure clause** for the mechanism and the measured numbers — it is the phase-close PPL gate this exit index points at.

8.9 Notes for the picking-up session (per backend)

CPU. Start from a blank `engine/cpu/forward.c`. Wire the GGUF loader first, get tensors materialised into row-major-by-token layout, then implement the 13 steps as 13 functions. Keep the AVX2 path beside a scalar reference under an `SP_CPU_SCALAR=1` env gate so the scalar path stays buildable. The AVX512 path is a second pass and not required to close E_CPU_4.

E_CPU_2 oracle (do NOT use the local llama.cpp checkouts). The `llama-cpp-*` builds under `D:\F\` and `D:\F\shannon-prime-repos\` are all contaminated — they link `shannon_prime_*` libs (even the one named "cleanroom") — so their logits are non-stock and they sit in the anti-contamination zone. The clean reference is a pristine upstream llama.cpp at `D:\F\shannon-prime-repos\shannon-prime-lattice-llama` with the `dump_logits` tool at `shannon-prime-system-engine/tools/oracle/`; it dumps token IDs + per-position logits so the engine validates on identical IDs. Same rule applies to every later backend's correctness gate.

CUDA. Mirror the CPU layer-by-layer. The most fragile cell is the fused matmul + Frobenius decompress kernel — get the scalar correctness right via Ninja `--use-local-env` before turning on cublas LT or any tensor-core fusion. Output verification compares against the CPU run for the same prompt, so keep the CPU build alive in the same Phase 2-CU sessions.

Vulkan. Compile glslc shaders into SPIR-V at build time, not at runtime; the JIT path has caused at least one driver crash in prior attempts. Subgroup ops vary by vendor; test under at least two vendors before declaring 2-VK closed. MoltenVK on Apple Silicon is the canonical "third vendor" the test harness picks up later.

Hexagon. Build environment first. Do not attempt to bring up the forward pass until `env-hexagon.bat` produces a working `qaic.exe` invocation per the `reference_hexagon_build_recipe` memory. The FastRPC silent-fallback bug (`project_hexagon_silent_fallback`) is load-bearing context — when in doubt, log every rpcmem registration size and grep the logs against the IDL parameter sizes.

---

9. Phase 3 — Model-family expansion

Goal. Every backend supports every model family in the in-scope list. The matrix is large (7 model families × 4 backends = 28 cells), and not every cell must close — some cells are explicit "later" cells (see priority below).

Dependencies. Phase 2-CPU closed. Other backends may or may not be closed; the matrix is filled per-cell.

Parallelism. Cells are independent. Sessions can pick any cell.

9.1 Model-family list

- 3.1 Llama 3.1 / 3.2 — established baseline; Llama 3 is the most-tested upstream architecture, so it exercises every loader edge case.
- 3.2 Qwen3 base — closest to the current canonical Qwen3-0.6B test model.
- 3.3 Qwen3.5 — incremental update on Qwen3, mostly architecture-flag differences.
- 3.4 Qwen3.6 — **MoE.** Adds the routing layer, sparse FFN, expert parameter sharding. Largest single-cell scope in Phase 3.
- 3.5 Qwen3.7 — incremental update on 3.6 if it lands by then; otherwise deferred.
- 3.6 Gemma 2.5 / 3 / 4 — Google family. Different RoPE shape, different RMSNorm placement, attention sliding window. Gemma 3 is the canonical research target (the T4 validation curve was on Gemma3-1B).
- 3.7 DeepSeek V4 — large MoE with FP8 weights. Lower priority for initial bring-up; the architecture is heavy and the FP8 weights would require an additional dequant path.

9.2 Priority cells

The matrix has 28 cells. To stay honest about scope, the project declares which cells **must** close by end of Phase 3 and which are later-phase:

- **Must close (8 cells).** CPU × {Llama 3.x, Qwen3, Gemma 3}, CUDA × {Llama 3.x, Qwen3, Gemma 3}, Hexagon × {Qwen3, Gemma 3}.
- **Should close (10 cells).** CPU × {Qwen3.5, Qwen3.6, Gemma 2.5, Gemma 4, DeepSeek V4}, CUDA × {Qwen3.5, Qwen3.6, Gemma 2.5, Gemma 4, DeepSeek V4}.
- **Later (10 cells).** All Vulkan cells beyond the canonical Qwen3-0.6B test model. All Hexagon cells beyond Qwen3 and Gemma 3.

9.3 Per-cell deliverables

Each cell delivers:

- `engine/models/<family>/loader.<backend>.c` — model-specific loader.
- `engine/models/<family>/forward.<backend>.c` — forward pass.
- `engine/models/<family>/test_<backend>.c` — correctness tests.
- An entry in `engine/models/matrix_status.md` updated when the cell closes.

9.4 Per-cell tests

- M___1 — model loads.
- M___2 — first 256 tokens of forward pass match llama.cpp within 1e-3 relative.
- M___3 — PPL on a standard 4k-token sample within 0.5% of baseline.
- M___4 — memory footprint reported in the offload note.

Cells close on M_4 passing. A failed M_2 indicates a loader or forward-pass bug; do not declare the cell closed by skipping it.

9.5 Phase 3 exit

Phase 3 closes when the 8 must-close cells are green and the should-close cells are at least started. Tag `lat-phase-3-closed`. Later cells continue to land in Phase 4+ alongside their own work.

9.6 Architectural notes per model family

Llama 3.1 / 3.2. Most-tested upstream architecture, which means the loader exercises every GGUF metadata edge case the project will encounter. Use Llama 3.x as the canary when adding any new loader field. Llama's RoPE is the "plain" reference shape; everything else in the family list is a delta against it.

Qwen3 base. Closest to the canonical test model and the easiest cell to bring up second. The Qwen3 RoPE includes a per-head linear bias which the Stern-Brocot variant (E9.1) wires into directly; keep both code paths buildable until Phase 4 has Stern-Brocot validated end-to-end on this family.

Qwen3.5. Incremental delta on Qwen3 base. Most of the bring-up cost is metadata flags and a slightly different normalisation order inside the attention block; the kernel inventory is identical.

Qwen3.6 (MoE). This is the architectural step change inside the Qwen family. The routing layer assigns each token to k-of-N experts and the FFN becomes a sparse gather. The cell must add an expert parameter sharding path so the experts can be split across multiple machines or pipeline stages later. Phase 3 only requires the single-machine case; multi-machine MoE is a later phase.

Qwen3.7. Speculative. If 3.7 has shipped by the time the project gets here, treat it as another incremental Qwen update; if it hasn't, defer this row.

Gemma 3 (and 2.5 / 4). The canonical research + PPL target — T_FRO_4 runs on Gemma3-1B, and it is the first second-architecture both 2-CPU and 2-CU closed. Architecture deltas vs the Qwen/Llama base, all exercised by `gemma3_forward` (`src/forward/gemma3.c`) and `gemma3_forward_cuda`: embedding scaled by √n_embd; **sandwich RMSNorm** — a post-attention and a post-FFN norm on the residual branch in addition to the pre-norms (the `(1+w)` is baked into the GGUF weights at conversion, so the norm is the plain `x/rms·w`, NOT `(1+w)` — adding 1 double-counts); per-head QK-RMSNorm over head_dim *before* RoPE; **dual RoPE base + sliding-window attention** — a `set_swa_pattern(6)` schedule alternates local layers (sliding window 512, base 10000) and global layers (full causal, base 1e6), global iff L%6==5 (4 global / 22 local of 26 on 3-1B); **GeGLU** FFN (gelu-tanh(gate)·up), not SwiGLU; tied LM head; no final-logit softcap on 3-1B. Gemma 2.5 / 4 are incremental deltas on this shape (4 adds a vision tower the text path ignores). Per-cell gate = the same distributional + PPL checks as the reference model (`M_GEMMA3_*`, T_FRO_4).

---

10. Phase 3-HX-MODE-C — HTP-augmented Hexagon backend

The Mode B baseline (Phase 2-HX) closes T_FRO_4 on HVX-only kernels. Mode C layers a QNN HTP dispatch on top: the heavy QK^T matmuls run on the V69 Hexagon Tensor Processor while the FFN stays on HVX. The two execute in parallel — HTP on layer N+1's QK^T while HVX is computing FFN of layer N.

Dependencies. `lat-phase-2-hx-closed` (Mode B baseline). All six E_HX gates green, including T_FRO_4 split-gate.

10.1 Deliverables

- **E_HXC_1 — QNN HTP runtime initialization.** `libQnnHtp.so` loaded at session create; QnnGraph created with the model's attention head dimensions baked in; weights uploaded to the HTP's local memory at `sp_model_load` time (not per-step). Reference: existing 2-CU work on QNN HTP runtime graphs (`project_phase25_runtime_graph_validated`) is the closest precedent but cannot be copied per anti-contamination.
- **E_HXC_2 — QK^T HTP dispatch in `sp_decode_step`.** The attention kernel calls into the HTP for QK^T (and only QK^T — softmax, mask, V-sum stay on HVX). HTP receives Q and K in SVM ION buffers (already allocated at session create); writes the score matrix back to ION; HVX picks up. Cost target: HTP dispatch ≤ 100 µs per layer on Gemma3-1B.
- **E_HXC_3 — HTP/HVX overlap correctness.** While HVX is doing FFN of layer N, HTP is doing QK^T of layer N+1. Synchronization via a FastRPC semaphore. Gate: 100-step decode produces bit-identical logits between Mode B (no overlap) and Mode C (with overlap) — proves the parallel execution is race-free.
- **E_HXC_4 — T_FRO_4 split gate on Mode C.** Same gate as Mode B with `SP_HX_MODE=C` engaged: (a) engine-hexagon-C-f32 vs engine-cpu-f32 ≤ 0.05% PPL drift; (b) per-row Q8 drift ≤ 2%.

10.2 Build env

Existing `scripts/env/env-hexagon.bat` plus QNN SDK pin — documented in BUILD-ENV.md once 2-HX (Mode B) closes and the agent can write authoritatively about the QNN dependency.

10.3 Anti-contamination

The old cohort's QNN integration lives in `D:\F\shannon-prime-repos\shannon-prime-engine\src\qnn\` and the related `project_phase25_qnn_in_llama_cli` work was explicitly RETRACTED per memory. The Mode C agent will reimplement QNN dispatch fresh inside `shannon-prime-system-engine/src/backends/hexagon/qnn/`. Reference the prior work for what NOT to do (the runtime gate that blocked engagement); do not copy code.

10.4 Exit

E_HXC_1..4 green. Tag `lat-phase-3-hx-mode-c-closed` on engine + system. SESSION-STATE entry names dispatch latencies, overlap correctness numbers, and the Mode B vs Mode C wall-clock delta.

---

11. Phase 3-HX-MODE-D — ISP-augmented Hexagon backend

The Mode C baseline (Phase 3-HX-MODE-C) overlaps HTP and HVX. Mode D adds a third compute resource: the Spectra 680 ISP runs the fused FFN at 18-bit fixed-point via Halide AOT-compiled kernels. ISP + HTP + HVX all run in parallel — ISP on FFN layer N, HTP on QK^T layer N+1, HVX on residual fixup + norms.

Dependencies. `lat-phase-3-hx-mode-c-closed`. The Mode C parallel-dispatch correctness gate (E_HXC_3) is what makes the three-way overlap of Mode D tractable.

11.1 Deliverables

- **E_HXD_1 — Halide generator + AOT compilation.** Halide C++ generator emits per-arch `ffn_skeleton_<arch>.a` static archives (one per arch_id: llama3, qwen3, gemma3, deepseek_v4). Each archive carries its own activation polynomial — HardSwish-SwiGLU for SwiGLU archs, piecewise polynomial GeGLU for Gemma3 (per Systems Appendix C §C.3 / §C.4). Halide schedule uses `compute_at(ffn_out, xi)` (NOT `compute_at(ffn_out, x)`) to keep accumulation inside the vectorized inner loop.
- **E_HXD_2 — IDL + FastRPC bridge.** `ffn_fusion.idl` declares `run_ffn_skeleton` with `rout` (not `inout`) for the output buffer — saves the copy-in of uninitialized state. `rpcmem_alloc` sizes match IDL *Len parameters exactly (per `feedback_fastrpc_exact_alloc`). Scales are pre-converted to int32 Q-point at session create time and passed as `int32*` (NOT `float*`) over the bus.
- **E_HXD_3 — Per-arch activation parity.** For each arch: E_HX_D_SWIGLU_KL ≤ 2e-3 vs f32 SwiGLU oracle (Llama 3.x, Qwen 3, DeepSeek V4); E_HX_D_GEGLU_KL ≤ 2e-3 vs f32 GELU-tanh oracle (Gemma 3). Gate fails if HardSwish formula is missing the `· gate / 6` term (the documented early-implementation bug per Appendix C §C.3).
- **E_HXD_4 — Three-way parallel correctness.** A 100-step decode with ISP + HTP + HVX all engaged produces bit-identical logits to Mode C (HTP + HVX only). Proves the ISP dispatch doesn't race against the other two.
- **E_HXD_5 — Thermal-pause auto-tune.** Default `thermal_pause_us=1500` on the S22U. The agent profiles to confirm the value rides the firmware's throttle limit without triggering it during a 5-minute sustained decode. Records the empirical value in BUILD-ENV.md.
- **E_HXD_6 — T_FRO_4 split gate on Mode D.** With `SP_HX_MODE=D` engaged: (a) engine-hexagon-D-f32 vs engine-cpu-f32 ≤ 0.05% PPL drift; (b) per-row Q8 drift ≤ 2%. May tighten the gate to account for the 18-bit fixed-point activation precision floor; document as a §8.6.1-style distributional gate with explicit KL bound (target: ≤ 5e-5 vs Mode C; the 18-bit precision floor is approximately one decimal digit looser than HVX's int32).
- **E_HXD_7 — Two new sp_status codes wired through.** `SP_EHX_ISP_DISPATCH` and `SP_EHX_THERMAL_TRIP` per Appendix C §C.8. L2's error-handling for `SP_EHX_THERMAL_TRIP` is a soft retry with bumped `thermal_pause_us`; that retry logic lives in the Rust engine driver (L2), not in L1.

11.2 Build env

- Halide ≥ 17.0 on the Windows host (host-side AOT compilation).
- Hexagon SDK 5.4.0.x with `hexagon-clang` for DSP-side compile.
- Git `sh.exe` in PATH (per `reference_hexagon_build_recipe`).
- `qaic.exe` from `WinNT/` subdirectory (per same memory).
- Reference design (Halide generator, IDL, CMake glue, `deploy-s22u.bat`): `papers/MODE_D_DESIGN_DRAFT.md`.

11.3 Anti-contamination

The old cohort's Halide work lives at `D:\F\shannon-prime-repos\shannon-prime-llama\backends\halide\` and the related `reference_halide_windows_build` memory. The freethedsp shim lives at the same path under `backends/freethedsp/`. **Semantic** patterns carry forward (Halide AOT to static archive, hexagon-clang for DSP side, LD_PRELOAD for unsigned PD, ADSP_LIBRARY_PATH trailing semicolon, 1500µs thermal pause). **Code** must be reimplemented fresh inside `shannon-prime-system-engine/src/backends/hexagon/halide/` and `shannon-prime-system-engine/src/backends/hexagon/fr`

11.4 Exit

E_HXD_1..7 green on the S22U with all per-arch activation parity gates passing. Tag `lat-phase-3-hx-mode-d-closed`. SESSION-STATE entry includes: per-arch KL drift numbers, ISP dispatch latency, three-way parallel correctness, sustained-decode thermal profile, and the empirical `thermal_pause_us` for the S22U.

11.5 Non-goals for Mode D v0

- On-chip Snapdragon X Elite / 8 Gen 2 / 8 Gen 3 variants. v0 targets S22U (Snapdragon 8 Gen 1) only because that's the validated hardware. Newer generations land as Phase 4+.
- ISP for non-FFN ops. v0 routes only the fused FFN through the ISP. QK^T and attention stay on HTP / HVX even though the ISP could in principle do them too — adding ops to the ISP path multiplies the schedule-tuning surface and is deferred until the FFN path proves stable.
- Halide JIT compilation on-device. v0 is AOT-only; the per-arch static archives ship in the engine library. Runtime JIT requires Halide runtime on Android which we don't pay for.

Phase log

One paragraph per closed phase (§3.3). Most recent last.

Phase 1 — Math core foundations — closed all tiers (2026-05-21)

All six subphases green. **1A** O_K arithmetic over Q(√-163) (`core/ok_arith`, T_OK_1..6). **1B** dual-prime CRT negacyclic NTT (`core/ntt_crt`, T_NTT_1/2/4/5; N∈{128,256,512} on the frozen primes; no `__int128` in the production path, configure-time guard). **1C** R_q polynomial-ring attention (`core/poly_ring`, T_PR_1..4; ⟨q,k⟩ recovered exactly via the nega-cyclic involution, T_PR_2 KL=0). **1D** VHT2 + Möbius + frozen 63-byte Spinor block (`core/vht2`, T_VHT_1..6). **1E** Frobenius Q8 lift (`core/frobenius`, T_FRO_1..3; T_FRO_4 → Phase 2). **1F** KSTE encoder + Tier-0/Tier-1 dominance (`core/kste`, T_KSTE_1..5; frozen 64-byte T_{60,3} tree). Built scaffold-first (`cmake/sp_module.cmake`, `sp_test.h`, EXISTS-guarded root) and executed by parallel agent dispatch. Integrated `ctest` 6/6, UBSan-clean on Windows MinGW-gcc (Tier-1); Linux gcc CI green (Tier-2). **Tier-3 (MSVC) also closed same session:** full suite 6/6 under MSVC (VS2019 BT), T_NTT_3 gated against a gcc-pre-generated `__int128`-oracle fixture (`ntt_ref_vectors.h`, cases to 2^28 exercising the ~2^60 CRT range), T_VHT_5 / T_KSTE_4 byte-identity green under MSVC, and a `windows-msvc` CI job added so all three tiers stay continuously gated. Scaffold gained `sp_add_module(... DEPENDS ...)` for inter-module links. **Still open:** T_FRO_4 (Gemma3-1B PPL) runs in Phase 2. Tag: `lat-phase-1-closed`. Offload: `SESSION-CLOSED-lat-1.md`.

Phase 2-CPU — canonical engine backend — closed (2026-05-22)

CPU forward pass green end-to-end on the reference model **and** on Gemma3-1B. **E_CPU_1..6** — GGUF loader, distributional forward vs the clean llama.cpp oracle (§8.6.1), Frobenius/Q8 lift, AVX2, NTT-attention (KL 2.7e-10 ≤ T_PR_2), KSTE KV signatures. **Foundational compression** (§8.2.1) — E_CPU_7 Q4 inline weights, E_CPU_8 VHT2+Spinor KV codec, GEN_KV persistent-KV O(n) decode. **Load-bearing layout** (§8.2.2) — E_CPU_9 packed-weight arena (Q8 574.5 MB / Q4 300.7 MB, byte-identical to `SP_ENGINE_FROB=1/3`), E_CPU_10 F16-source release, persistent Spinor KV (2.71×), COMPOSE + format-lock `_Static_asserts`; the arena + KV-split primitives ported into the math core (`core/frobenius` T_FRO_5, `core/vht2` T_VHT_7) so all backends read one byte contract (`SP_FROB_ARENA_LAYOUT_VERSION=1`, `SP_SPINOR_LAYOUT_VERSION=1`). **Gemma3-1B SP1** — 2nd architecture (SentencePiece tokenizer `SPM_ENCODE`; sandwich norms,

GeGLU, local/global sliding-window attention, dual RoPE base, tied head); the f32 forward distributionally matches the oracle (M_GEMMA3_CPU: argmax 6/6, top-5 6/6, mean KL 1.65e-6). **T_FRO_4** (the §8.2 closure clause) closes the phase: engine-f32 PPL vs the f16 oracle **−0.0146%** (gate ≤ 0.05%), per-row Q8 arena drift **−0.74%** (gate ≤ 2%) — the original single 0.1% target was split because the production arena is per-row (~1% lossy by design, E_CPU_3). Full CPU regression **20/20 green** incl. T_FRO_4 (SLOW). Engine commits through 3431cf8 (SP2 41c5e19, SP3 3431cf8). This roadmap's §8.2–§9 + Phase log were restored here after the Three-Gap-integration commit (6fba9e2) truncated them. Other backends (2-CU/VK/HX, §8.3–8.5) remain open. Tag: lat-phase-2-cpu-fro4-closed. Offload: SESSION-CLOSED-lat-2-CPU.md.

2026-05-23 — Phase 2-FMT closed (lat-phase-2-fmt-closed)

sp_model_load (mmap zero-copy header parse, ≤250 LOC) + sp-transcode CLI (GGUF → .sp-model with per-row Frobenius Q8 + Spinor-format-lock adherence + sibling-tensor spatial-locality) + .sp-tokenizer extraction (128-byte header + raw SentencePiece/BPE blob, SHA-256 paired against model header) + Appendix B §12.3 round-trip gate all green. Engine commits 1cfb85a2..3e553fcc on the lat-2-FMT branch.

E_FMT_4 on Gemma3-1B (transcode → load → gemma3_forward bit-identical vs GGUF arena-q8): bit_exact=YES, worst_abs=0, L2-drift=0.000000%, argmax 8/8 PASS. E_FMT_4_QWEN3 cross-arch on Qwen3-0.6B: identical numbers. Format is round-trip-safe on both reference architectures.

Offload: SESSION-STATE-lat-2-FMT.md.

2026-05-23 — Phase 2-VK closed (lat-phase-2-vk-closed,

lat-phase-2-vk-fro4-closed)

T_FRO_4_VK split gate GREEN: (a) vulkan-f32 PPL 32.86458 vs oracle f16 PPL 32.86939, rel-diff −0.0146% (gate 0.05%) PASS; (b) per-row Q8 arena PPL 32.62294, drift −0.7353% (gate 2%) PASS. Mirrors 2-CU exactly on Vulkan compute. SPIR-V shaders glslc-compiled at build time. f64 accumulator in matmul keeps Vulkan logits at the f32 floor of CPU (KL ~1e-11). KSTE host-encode 3-part gate green per reference-kste-cross-backend-gate.

Engine HEAD 0c243eca / merge 99ccb04d on the lat-2-VK branch. Offload: SESSION-STATE-lat-2-VK.md.

2026-05-23 — Phase 2-HX essentially closed (HVX-accelerated Gemma3

forward on V69 GREEN; formal close tag pending E_HX_5/E_HX_6)

HX.0 env+toolchain hard-gate → HX.1 aarch64-android cross-compile + on-phone CPU PPL −0.0144% baseline → HX.2-prep 574 MB rpcmem capacity probe (single-buffer Q8 arena feasible) → HX.2 FastRPC IDL round-trip on device (sp_hex_ping(41)→42(rc=0x0), unsigned PD domain 3) → HX.3a step 1 per-tensor rpcmem upload byte-exact (host CRC-32 == DSP CRC-32) → HX.3a step 2 cDSP scalar f32 matmul bit-exact to host (worst |dsp-host| = 0) → HX.3a forward green (scalar f32 gemma3 layers on cDSP match CPU Q8 at KL 9.19e-11) → HX.3b HVX matmul on V69 + HVX gemma3 forward (worst_rel 7.9e-5, KL mean 8.9e-11).

T_FRO_4_HX met transitively: on-phone hexagon-Q8 PPL 32.62290 vs f16 oracle 32.86939 → −0.75% (gate 2%) PASS — exactly the cross-backend Q8 PPL. The hex forward is Q8-arena-only by design (no hex-f32 path); gate (a) inherits the HX.1 on-phone CPU-f32 baseline.

Engine commits through 654c6a68 on the lat-2-HX branch. Remaining for the formal lat-phase-2-hx-closed tag: E_HX_5 (host int64-dot identity test — already proven on CPU) +

E_HX_6 (host sp_kste_encode 3-part gate — needs a D→H copy of cDSP post-norm K). Bounded continuation.

Offload: SESSION-STATE-lat-2-HX.md.

2026-05-23 — Phase 2-L1.RELOCATE closed (twelve increments, math-core consolidation)

The entire reference inference path migrated from engine-side into the math core via twelve gated commits on shannon-prime-system's lat-l1 branch:

1. ABI contract surface + reference forward kernels (8c228e86)
2. .sp-model hash primitives (c7958459)
3. .sp-model format/loader layer — core/io_format (cced325e)
4. GGUF weight-dtype dequant (bd601633)
5. GGUF v3 model parser (3fdf874b)
6. Model-representation header (5cafbb16)
7. Packed-weight arena — T_ARENA 14/14 (8167b34b)
8. GGUF load/free/release lifecycle — T_MODEL (5c6dc72b)
9. Model-coupled weight-lift kernels — T_FWD_DISPATCH, bit-exact parity (c99cb301)
10. Qwen3 forward orchestration — T_FORWARD, end-to-end real-model smoke (61557f95)
11. Gemma3 reference forward — T_FORWARD, end-to-end smoke (eed45361)
12. Qwen3 KV-decode + greedy generate — T_FORWARD 2/2 (222e252c)

System HEAD 222e252c. The engine still has its (now-redundant) copies of the relocated files; the engine's regression suite has NOT yet been re-run against the math-core sources. That is the explicit gate of sub-phase 8.7.2 (Phase 2-L1.VALIDATE) — the only thing that proves the relocations are bit-exact.

Closure tag deferred to the umbrella lat-phase-2-l1-closed after sub-phases 8.7.2 / 8.7.3 / 8.7.4 close in order.

---

20. Research Track — φ-RoPE / Three-Gap frequency-sort restructuring

Demoted from the Phase 2 critical path 2026-05-22 after the Phase 2-CPU agent identified that the original 2-B.E.1 framing required a precondition (linear-in-φ RoPE frequencies) that stock pretrained models do not satisfy. The cyclotomic-ring polynomial-shift cache (new 2-B.E.1) delivers a strictly larger lossless win (~128×) on stock models, so the frequency-sort restructuring is no longer competing for the same slot.

The items below remain interesting as research investigations against models that ship with φ-derived (linear-in-φ) RoPE — none currently exist in the engine's target families, so these are speculative future work. They are NOT prerequisites for any Phase 2..13 deliverable.

20.1 φ-RoPE schedule swap

Replace geometric $\theta_d = \text{base}^{-2d/D}$ with linear $\theta_d = \{d\varphi\} \cdot 2\pi$. This is a positional-basis change; on stock models it degrades long-context quality. Production use requires the model to be pretrained or fine-tuned on the new schedule. Validation gate would be a PPL-drift sweep on a calibration corpus (mirroring Phase 4 Fibonacci-KV). Gate SP_ROPE_PHI=1; currently parked.

20.2 Three-Gap frequency-sort cache restructuring (lossless under φ-RoPE)

Given the §20.1 precondition (linear-in-φ frequencies), the relative-attention phase shifts across dimensions exhibit at most three distinct adjacent gaps (Theorem T7 corollary). The rotation cache collapses to $O(\text{ctx} \cdot 3)$ entries at D dimensions — roughly $D/3 \approx 43\times$ reduction at $D = 128$. Lossless given the precondition; pointless without it. Gate `SP_ROPE_3STEP_CACHE=1`; no-op when `rope.style` is not `phi`.

20.3 Trigger conditions for revisiting

These items move back onto a numbered phase when **any** of:

- A φ-RoPE-pretrained model lands in one of the engine's target families (Llama, Qwen, Gemma, DeepSeek);
- A fine-tuning run on a stock model with a φ-RoPE schedule produces PPL within 1% of the original on a calibration corpus (validating that the schedule swap is recoverable);
- An external research result demonstrates that the geometric → linear-in-φ swap can be done at inference time without retraining (currently no such result exists).

Until one of these triggers fires, the production rel-attn cache is the §2-B.E.1 polynomial-shift cache (lossless on stock RoPE, gated by `SP_ENGINE_NTT_ATTN=1`), and the §20 items remain parked research notes — off the Phase 2..13 critical path.
